transportation: the Transportation program, user guide

transportation-ai999x 1.1.0, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Transportation covers vehicle dynamics, braking and stopping distances, traffic flow, highway geometry, railway operations, route planning and transit capacity. This guide is about `transportation`, the model as a self-contained program for Windows and Linux: 77 capabilities in 13 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/transportation-ai999x/); the Python package (`pip install transportation-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.5.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://downloads.ai999x.com/desktop/1.5.0/SHA256SUMS; the newest version is always under https://downloads.ai999x.com/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Transportation	Windows x86-64	https://downloads.ai999x.com/desktop/1.5.0/ai999x-transportation-1.5.0-windows-x86_64.zip
Desktop app: Transportation	Linux x86-64	https://downloads.ai999x.com/desktop/1.5.0/ai999x-transportation-1.5.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://downloads.ai999x.com/desktop/1.5.0/ai999x-1.5.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://downloads.ai999x.com/desktop/1.5.0/ai999x-1.5.0-linux-x86_64.tar.gz
Command line	Windows x86-64	https://downloads.ai999x.com/desktop/1.5.0/ai999x-cli-1.5.0-windows-x86_64.zip
Command line	Linux x86-64	https://downloads.ai999x.com/desktop/1.5.0/ai999x-cli-1.5.0-linux-x86_64.tar.gz

Desktop app

Transportation in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-transportation-1.5.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-transportation.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.5.0/ai999x-transportation-1.5.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.5.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-transportation-1.5.0-linux-x86_64.tar.gz
./ai999x-transportation-1.5.0-linux-x86_64/ai999x-transportation
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.5.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.5.0/ai999x-1.5.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.5.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.5.0-linux-x86_64.tar.gz
./ai999x-1.5.0-linux-x86_64/ai999x
```

Command line, Linux

The bundle holds one program per model; `transportation` is this one. The checksum line should print OK.

```
curl -LO https://downloads.ai999x.com/desktop/1.5.0/ai999x-cli-1.5.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.5.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.5.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.5.0-linux-x86_64:$PATH"' >> ~/.bashrc && source
~/.bashrc
transportation --version
```

Command line, Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.5.0-windows-x86_64.zip"
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.5.0/$F" -OutFile $F
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.5.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F
SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.5.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User")
+ ";$D", "User")
transportation --version
```

In your browser

Open https://www.ai999x.com/models/transportation-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install transportation-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ transportation --version
transportation-ai999x 1.5.0 (Rust build, ax-engine 1.5.0)
```

2.2 Ask in words

Write the question as you would say it: the program reads the quantities and their units, picks the capability and answers.

```
$ transportation ask "Stopping sight distance with speed = 100 km/h, reaction time = 2.5 s and
deceleration = 3.4 m/s^2."
Read as: Stopping sight distance (reaction plus braking, with grade)
SSD = 182.9157589 m
d_r = 69.44444444 m
d_b = 113.4713145 m
  by: Stopping sight distance (reaction plus braking, with grade)  (SSD = v t_r + v^2 / (2 g (f +
G)), or with a deceleration rate v t_r + v^2 / (2 (a + g G)))
  a = 3.4 m/s^2
  t_r = 2.5 s
  v = 27.77777778 m/s
  assumes: v > 0; t_r >= 0; exactly one of f, a; f + G > 0 (or a + g G > 0)
```

2.3 Find a capability

Search the capabilities by what you want to know; the first column is the id to pass to info and solve.

```
$ transportation search "queue and uniform delay at a signal (deterministic d/d/1)"
transport_signal_queue_uniform_delay Queue and uniform delay at a signal (deterministic D/D/1)
(traffic signals)
transport_md1_queue                  Single server with constant service time (M/D/1)  (queueing)
transport_bottleneck_queue           Queue and delay at a temporary bottleneck (incident, lane
closure)  (traffic flow)
transport_signal_queue_shockwave     Queue at a signal by shockwave analysis  (traffic signals)
transport_interval_queue             Queue from arrival counts and capacity per interval  (traffic
flow)
transport_fifo_queue_simulation      First-come-first-served queue from arrival and service times
(queueing)
transport_mm1_queue                  Single-server random queue (M/M/1), e.g. a toll booth
(queueing)
transport_mmc_queue                  Multi-server random queue (M/M/c, Erlang C), e.g. a toll plaza
(queueing)
… 12 more
```

2.4 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ transportation list
transport_acceleration_distance          Distance and time to accelerate (acceleration lane length)
transport_acceleration_under_power       Acceleration time and distance under constant engine power
transport_all_or_nothing_assignment      All-or-nothing traffic assignment
transport_bottleneck_queue               Queue and delay at a temporary bottleneck (incident, lane
closure)
transport_braking_distance               Braking distance from speed and deceleration
transport_braking_weight_transfer        Longitudinal weight transfer under braking and ideal brake
proportioning
transport_braking_with_air_resistance    Theoretical stopping distance with aerodynamic and rolling
resistance
transport_clothoid_coordinates           Clothoid (spiral) transition curve coordinates
… 69 more
```

2.5 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ transportation info transport_signal_queue_uniform_delay
Queue and uniform delay at a signal (deterministic D/D/1)
  Q = lam r, t_c = lam r/(s - lam), D = lam r^2 / (2(1 - lam/s)), d = r^2 / (2C (1 - lam/s))
  in   lam      arrival flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')
[1/s]
  in   s        saturation flow (veh/h in questions; veh/s in solve(), or a string such as '600
1/h') [1/s]
  in   r        effective red [s]
  in   g        effective green [s]
  out  Q_max    largest queue (veh) []
  out  t_clear  time after the start of green to clear the queue (s) [s]
  out  D_total  total delay per cycle (veh s) [s]
  out  d_uniform average delay per vehicle (s) [s]
  out  P_stopped share of vehicles that stop []
```

2.6 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ transportation solve transport_signal_queue_uniform_delay "lam=600 1/h" "s=1800 1/h" "r=40 s"
"g=30 s"
Q_max = 6.666666667
t_clear = 20 s
D_total = 200 s
d_uniform = 17.14285714 s
P_stopped = 0.8571428571
  by: Queue and uniform delay at a signal (deterministic D/D/1)  (Q = lam r, t_c = lam r/(s - lam),
D = lam r^2 / (2(1 - lam/s)), d = r^2 / (2C (1 - lam/s)))
  lam = 0.1666666667 1/s
  s = 0.5 1/s
  r = 40 s
  g = 30 s
  assumes: lam < s; the queue clears within the green (lam C <= s g)
```

2.7 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ transportation solve transport_signal_queue_uniform_delay "lam=600 1/h" "s=1800 1/h" "r=40 s"
"g=30 s" --json
{"outcome":"SOLVED","message":"","value":6.66666666667,"unit":"","solved_for":"Q_max","si":
[["lam",0.16666666666666666],["s",0.5],["r",40.0],["g",30.0]],"values":[["Q_max",6.66666666667,""],
["t_clear",20.0,"s"],["D_total",200.0,"s"],["d_uniform",17.1428571429,"s"],
["P_stopped",0.857142857143,""]],"target":"Q_max","phrase":"largest queue (veh)","assumptions":
["lam < s","the queue clears within the green (lam C <= s g)"],"name":"Queue and uniform delay at a
signal (deterministic D/D/1)","equation":"Q = lam r, t_c = lam r/(s - lam), D = lam r^2 / (2(1 -
lam/s)), d = r^2 / (2C (1 - lam/s))"}
```

2.8 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ transportation solve transport_signal_queue_uniform_delay "lam=600 1/h" "s=1800 1/h" "r=40 s"
NEEDS_INPUT: transport_signal_queue_uniform_delay needs g (effective green). Example:
solve('transport_signal_queue_uniform_delay', lam='0.166667 1/s', s='0.5 1/s', r='40 s', g='30 s').
```

2.9 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | transportation batch

`cases.jsonl`:

```
{"key": "transport_signal_queue_uniform_delay", "inputs": {"lam": "600 1/h", "s": "1800 1/h", "r": "40 s", "g": "30 s"}}
{"key": "transport_curve_safe_speed", "inputs": {"R": "300 m", "e": "0.06 1", "f": "0.12 1"}}
```

```
$ transportation batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":6.66666666667,"unit":"","solved_for":"Q_max","si":
[["lam",0.16666666666666666],["s",0.5],["r",40.0],["g",30.0]],"values":[["Q_max",6.66666666667,""],
["t_clear",20.0,"s"],["D_total",200.0,"s"],["d_uniform",17.1428571429,"s"],
["P_stopped",0.857142857143,""]],"target":"Q_max","phrase":"largest queue (veh)","assumptions":
["lam < s","the queue clears within the green (lam C <= s g)"],"name":"Queue and uniform delay at a
signal (deterministic D/D/1)","equation":"Q = lam r, t_c = lam r/(s - lam), D = lam r^2 / (2(1 -
lam/s)), d = r^2 / (2C (1 - lam/s))"}
{"outcome":"SOLVED","message":"","value":23.0993893616,"unit":"m/s","solved_for":"v","si":
[["R",300.0],["e",0.06],["f",0.12]],"values":[["v",23.0993893616,"m/s"],
["v_kmh",83.1578017018,"km/h"]],"target":"v","phrase":"maximum safe speed (m/s)","assumptions":["R
> 0","f > 0","e f < 1"],"name":"Maximum safe speed on a superelevated horizontal
curve","equation":"v = sqrt(g R (e + f) / (1 - e f)) (point-mass equilibrium)"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `transportation info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]"` or `"v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`transportation solve ... --json` prints one answer as one line of JSON. `transportation batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | transportation batch`. `transportation catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]
assumptions	the conditions the answer relies on
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["transportation", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`transportation figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `transportation_ai999x.wasm` (https://www.ai999x.com/compute/rust/transportation_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("transportation_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `transportation info <id>` shows the same inputs and outputs in the terminal.

Braking and stopping (6)

Braking distance from speed and deceleration, `transport_braking_distance`

Distance and time for a vehicle to stop from speed v at a constant deceleration a (no reaction time).

$$d = \frac{v^2}{2a}, \quad t = \frac{v}{a}$$

symbol		meaning	SI unit
`v`	in	initial speed	m/s
`a`	in	deceleration (magnitude)	m/s^2
`d`	out	braking distance (m)	m
`t`	out	braking time (s)	s

Applies when: v > 0; a > 0.

```
$ transportation solve transport_braking_distance "v=27.77777778 m/s" "a=3.4 m/s^2"
d = 113.4713145 m
t = 8.169934641 s
  by: Braking distance from speed and deceleration  (d = v^2 / (2a), t = v / a)
  v = 27.77777778 m/s
  a = 3.4 m/s^2
  assumes: v > 0; a > 0
```

What the worked example means: This calculation determines the distance a vehicle travels to stop completely from an initial speed of 27.7778 m/s under a constant deceleration of 3.4 m/s^2. The resulting braking distance is 113.471 m, and the time required to reach a full stop is 8.16993 s. These values assume no driver reaction time and level ground.

Distance to slow from one speed to another (friction and grade), `transport_speed_reduction_distance`

Braking distance and time to slow from v1 to v2 with friction coefficient f on a grade G (deceleration g (f + G)).

$$d = \frac{v_1^2 - v_2^2}{2g(f + G)}$$

symbol		meaning	SI unit
`v1`	in	initial speed	m/s
`v2`	in	final speed	m/s
`f`	in	friction coefficient	,
`G`	in	grade, decimal (positive uphill) (optional)	,

symbol		meaning	SI unit
`d`	out	braking distance (m)	m
`t`	out	braking time (s)	s

Applies when: v1 > v2 >= 0; f + G > 0.

```
$ transportation solve transport_speed_reduction_distance "v1=25 m/s" "v2=11.11111111 m/s" "f=0.4
1" "G=0.03 1"
d = 59.44849938 m
t = 3.292532274 s
  by: Distance to slow from one speed to another (friction and grade)  (d = (v1^2 - v2^2) / (2 g (f
+ G)))
  v1 = 25 m/s
  v2 = 11.11111111 m/s
  f = 0.4
  G = 0.03
  assumes: v1 > v2 >= 0; f + G > 0
```

What the worked example means: This calculation determines the distance and time required for a vehicle to decelerate from an initial speed of 25 m/s to a final speed of 11.1111 m/s. The result assumes a constant deceleration governed by the friction coefficient 0.4 and the road grade 0.03. The computed stopping distance is 59.4485 m, which represents the length of road needed to reduce speed under these specific conditions, while the duration of this maneuver is 3.29253 s.

Impact speed with an obstacle after reaction and braking, `transport_impact_speed`

A driver at speed v sees an obstacle at distance D, reacts after t_r and brakes with friction f on grade G: the speed at the obstacle (0 if the vehicle stops in time) and the full stopping distance.

$$v_i = \sqrt{v^2 - 2g(f+G)(D - vt_r)}$$

symbol		meaning	SI unit
`v`	in	initial speed	m/s
`D`	in	distance to the obstacle	m
`t_r`	in	perception-reaction time	s
`f`	in	friction coefficient	,
`G`	in	grade, decimal (positive uphill) (optional)	,
`v_impact`	out	speed at the obstacle (m/s), 0 if the vehicle stops before it	m/s
`v_impact_kmh`	out	speed at the obstacle (km/h)	km/h
`d_stop`	out	stopping distance from the moment the obstacle is seen (m)	m

Applies when: v > 0; D > 0; t_r >= 0; f + G > 0.

```
$ transportation solve transport_impact_speed "v=25 m/s" "D=60 m" "t_r=1 s" "f=0.6 1"
v_impact = 14.59383431 m/s
v_impact_kmh = 52.53780353 km/h
d_stop = 78.09208291 m
  by: Impact speed with an obstacle after reaction and braking  (v_i = sqrt(v^2 - 2 g (f + G) (D -
v t_r)) when the obstacle lies within the braking phase)
  v = 25 m/s
  D = 60 m
  t_r = 1 s
  f = 0.6
  assumes: v > 0; D > 0; t_r >= 0; f + G > 0
```

What the worked example means: The impact speed is 14.5938 m/s (52.5378 km/h), with a total stopping distance of 78.0921 m.

Speed estimate from skid-mark length, `transport_skid_speed`

Initial speed of a vehicle that skidded to rest over a length d with friction coefficient f on a grade G.

$$v = \sqrt{2g(f + G)\,d}$$

symbol		meaning	SI unit
`d`	in	skid-mark length	m
`f`	in	friction coefficient	,
`G`	in	grade, decimal (positive uphill) (optional)	,
`v`	out	initial speed (m/s)	m/s
`v_kmh`	out	initial speed (km/h)	km/h

Applies when: d > 0; f > 0; f + G > 0.

```
$ transportation solve transport_skid_speed "d=30 m" "f=0.6 1"
v = 18.79255172 m/s
v_kmh = 67.65318618 km/h
  by: Speed estimate from skid-mark length  (v = sqrt(2 g (f + G) d))
  d = 30 m
  f = 0.6
  assumes: d > 0; f > 0; f + G > 0
```

What the worked example means: The estimated initial speed is 18.7926 m/s which converts to 67.6532 km/h.

Stopping sight distance (reaction plus braking, with grade), `transport_stopping_sight_distance`

Stopping sight distance = perception-reaction distance + braking distance on a grade, with either the longitudinal friction coefficient f (braking deceleration g (f + G)) or a deceleration rate a (AASHTO: a + g G, design value 3.4 m/s^2).

$$SSD = v\,t_r + \frac{v^2}{2g(f + G)}$$

symbol		meaning	SI unit
`v`	in	design speed	m/s
`t_r`	in	perception-reaction time	s
`f`	in	longitudinal friction coefficient (optional)	,
`a`	in	deceleration rate (optional)	m/s^2
`G`	in	grade, decimal (positive uphill) (optional)	,
`SSD`	out	stopping sight distance (m)	m
`d_r`	out	perception-reaction distance (m)	m
`d_b`	out	braking distance (m)	m

Applies when: v > 0; t_r >= 0; exactly one of f, a; f + G > 0 (or a + g G > 0).

```
$ transportation solve transport_stopping_sight_distance "v=27.77777778 m/s" "t_r=2.5 s" "a=3.4
m/s^2"
SSD = 182.9157589 m
d_r = 69.44444445 m
d_b = 113.4713145 m
  by: Stopping sight distance (reaction plus braking, with grade)  (SSD = v t_r + v^2 / (2 g (f +
G)), or with a deceleration rate v t_r + v^2 / (2 (a + g G)))
  v = 27.77777778 m/s
  t_r = 2.5 s
  a = 3.4 m/s^2
  assumes: v > 0; t_r >= 0; exactly one of f, a; f + G > 0 (or a + g G > 0)
```

What the worked example means: This calculation determines the total distance required for a driver to perceive a hazard, react, and bring the vehicle to a complete stop. The result, 182.916 m, is the sum of the perception-reaction distance, 69.4444 m, which depends on speed and reaction time, and the braking distance, 113.471 m, which accounts for deceleration capabilities and road grade. These values ensure that geometric design provides adequate visibility for safe operation at the given conditions.

Theoretical stopping distance with aerodynamic and rolling resistance, `transport_braking_with_air_resistance`

Stopping (or slowing) distance and time of a road vehicle braking with tyre-road friction mu and braking efficiency eta, helped by aerodynamic drag (rho C_D A v^2 / 2), rolling resistance f_r and the grade; gamma is the mass factor for rotating parts (1.04 for braking).

$$S = \frac{\gamma m}{2K} \ln \frac{F_0 + K v_1^2}{F_0 + K v_2^2}$$

symbol		meaning	SI unit
`m`	in	vehicle mass	kg
`v1`	in	initial speed	m/s
`mu`	in	tyre-road friction coefficient	,
`C_D`	in	drag coefficient	,
`A`	in	frontal area	m^2

symbol		meaning	SI unit
`rho`	in	air density (default 1.2 kg/m^3) (optional)	kg/m^3
`f_r`	in	rolling resistance coefficient (default 0.01) (optional)	,
`G`	in	grade, decimal (positive uphill) (optional)	,
`eta`	in	braking efficiency (default 1) (optional)	,
`gamma`	in	mass factor for braking (default 1.04) (optional)	,
`v2`	in	final speed (default 0) (optional)	m/s
`S`	out	braking distance (m)	m
`t`	out	braking time (s)	s

Applies when: m, v1, mu, C_D, A > 0; 0 < eta <= 1; v2 < v1; a positive constant retarding force.

```
$ transportation solve transport_braking_with_air_resistance "m=1500 kg" "v1=33.33333333 m/s"
"mu=0.7 1" "C_D=0.35 1" "A=2.2 m^2" f_r=0.012 "eta=0.95 1"
S = 84.8294941 m
t = 5.132823511 s
  by: Theoretical stopping distance with aerodynamic and rolling resistance  (S = gamma m / (2K)
ln((F0 + K v1^2) / (F0 + K v2^2)), K = rho C_D A / 2, F0 = W ((eta mu + f_r) cos(theta) +
sin(theta)))
  m = 1500 kg
  v1 = 33.33333333 m/s
  mu = 0.7
  C_D = 0.35
  A = 2.2 m^2
  f_r = 0.012
  eta = 0.95
  assumes: m, v1, mu, C_D, A > 0; 0 < eta <= 1; v2 < v1; a positive constant retarding force
```

What the worked example means: The braking distance is 84.8295 m and the time is 5.13282 s.

Highway geometry (12)

Clothoid (spiral) transition curve coordinates, `transport_clothoid_coordinates`

Coordinates along the tangent (x) and offset from it (y), and the tangent deflection angle, at distance s along a clothoid transition of length L_s that ends at a circular curve of radius R (A^2 = R L_s).

$$x = \int_0^s \cos\frac{u^2}{2A^2}\,du, \quad y = \int_0^s \sin\frac{u^2}{2A^2}\,du, \quad \theta = \frac{s^2}{2RL_s}$$

symbol		meaning	SI unit
`R`	in	radius of the circular curve	m
`L_s`	in	spiral length	m
`s`	in	distance along the spiral (default L_s) (optional)	m
`x`	out	distance along the initial tangent (m)	m
`y`	out	offset from the tangent (m)	m
`theta`	out	tangent deflection angle (degrees)	deg

Applies when: 0 < s <= L_s; tangent angle <= 90 degrees.

```
$ transportation solve transport_clothoid_coordinates "R=300 m" "L_s=80 m"
x = 79.85789479 m
y = 3.551043118 m
theta = 7.639437268 deg
  by: Clothoid (spiral) transition curve coordinates  (x = s (1 - theta^2/10 + theta^4/216 - ...),
y = s (theta/3 - theta^3/42 + ...), theta = s^2 / (2 R L_s))
  R = 300 m
  L_s = 80 m
  assumes: 0 < s <= L_s; tangent angle <= 90 degrees
```

In the worked example, R = 300 m, L_s = 80 m go through the equation above and give x = 79.85789479 m, y = 3.551043118 m, theta = 7.639437268 deg. It applies only when 0 < s <= L_s; tangent angle <= 90 degrees: outside that it says what is wrong instead of returning a number.

Curve radius or speed by the simplified e + f formula, `transport_simplified_curve_formula`

The minimum radius R = v^2/(g (e + f)) for a speed, or the maximum speed sqrt(g R (e + f)) on a radius, by the simplified (small-angle) point-mass formula of horizontal curves (the exact form is minimum curve radius).

$$R = \frac{v^2}{g\,(e + f)}$$

symbol		meaning	SI unit
`e`	in	superelevation rate, decimal	,
`f`	in	side friction factor	,
`v`	in	design speed (give v or R) (optional)	m/s
`R`	in	curve radius (give v or R) (optional)	m
`R`	out	curve radius (m)	m
`v`	out	speed (m/s)	m/s
`v_kmh`	out	speed (km/h)	,

Applies when: -0.1 <= e <= 0.2; 0 < f <= 0.5; the speed or the radius (one of them); small-angle (simplified) form.

```
$ transportation solve transport_simplified_curve_formula "e=0.0006 1" "f=0.12 1" "v=27.77777778
m/s"
R = 652.1968296 m
v = 27.77777778 m/s
v_kmh = 100
  by: Curve radius or speed by the simplified e + f formula  (R = v^2/(g (e + f)); v = sqrt(g R (e
+ f)))
  e = 0.0006
  f = 0.12
  v = 27.77777778 m/s
  assumes: -0.1 <= e <= 0.2; 0 < f <= 0.5; the speed or the radius (one of them); small-angle
(simplified) form
```

What the worked example means: This calculation determines either the minimum required curve radius based on design speed, superelevation, and side friction, or the maximum permissible speed given a fixed radius. Using the simplified point-mass formula, it balances centrifugal force against gravity and friction. The result indicates that for the input values provided, the curve must have a radius of at least 652.197 m to safely accommodate a vehicle traveling at 27.7778 m/s, or conversely, a vehicle can travel up to 27.7778 m/s on a curve with radius 652.197 m. This ensures stability by accounting for both the banking of the road and tire grip.

Horizontal sightline offset on a circular curve, `transport_horizontal_sight_offset`

The clearance M needed from the centre of the inside lane to an obstruction (middle ordinate) so that a driver on a curve of radius R sees the sight distance S, both driver and object on the curve.

$$M = R\left(1 - \cos\frac{S}{2R}\right)$$

symbol		meaning	SI unit
`R`	in	radius of the inside lane centreline	m
`S`	in	sight distance	m
`M`	out	horizontal sightline offset (m)	m

Applies when: R > 0; 0 < S < pi R.

```
$ transportation solve transport_horizontal_sight_offset "R=300 m" "S=160 m"
M = 10.60360643 m
  by: Horizontal sightline offset on a circular curve  (M = R (1 - cos(S / (2R))))
  R = 300 m
  S = 160 m
  assumes: R > 0; 0 < S < pi R
```

What the worked example means: This calculation determines the minimum clearance needed on the inside of a circular curve to ensure adequate visibility. Using the radius of the inside lane centerline, given as 300 m, and the desired sight distance, given as 160 m, the algorithm computes the middle ordinate M. The resulting value, 10.6036 m, represents the distance from the center of the inside lane to any obstruction that would block the driver's view along the straight chord connecting the driver and the object.

Maximum safe speed on a superelevated horizontal curve, `transport_curve_safe_speed`

The highest speed at which a vehicle (point mass) can hold a curve of radius R with superelevation e and side friction factor f.

$$v = \sqrt{\frac{gR(e+f)}{1-ef}}$$

symbol		meaning	SI unit
`R`	in	curve radius	m
`e`	in	superelevation rate, decimal	,

symbol		meaning	SI unit
`f`	in	side friction factor	,
`v`	out	maximum safe speed (m/s)	m/s
`v_kmh`	out	maximum safe speed (km/h)	km/h

Applies when: R > 0; f > 0; e f < 1.

```
$ transportation solve transport_curve_safe_speed "R=300 m" "e=0.06 1" "f=0.12 1"
v = 23.09938936 m/s
v_kmh = 83.1578017 km/h
  by: Maximum safe speed on a superelevated horizontal curve  (v = sqrt(g R (e + f) / (1 - e f))
(point-mass equilibrium))
  R = 300 m
  e = 0.06
  f = 0.12
  assumes: R > 0; f > 0; e f < 1
```

What the worked example means: This calculation determines the highest velocity a vehicle can maintain on a curve without skidding. It uses the curve radius, superelevation rate, and side friction factor to balance centrifugal forces against gravity and tire grip. The result shows that with a radius of 300 m, a superelevation of 0.06, and a friction factor of 0.12, the safe limit is 23.0994 m/s. This value helps engineers set appropriate speed limits for curved road sections.

Minimum length of a crest vertical curve for a sight distance, `transport_crest_curve_length`

The shortest crest vertical curve between grades g1 and g2 that provides the sight distance S (AASHTO: eye height 1.08 m, object height 0.60 m unless stated), choosing the S < L or S > L form.

$$L = \frac{AS^2}{200\left(\sqrt{h_1}+\sqrt{h_2}\right)^2} \ (S < L), \qquad L = 2S - \frac{200\left(\sqrt{h_1}+\sqrt{h_2}\right)^2}{A} \ (S > L)$$

symbol		meaning	SI unit
`g1`	in	grade before the curve, decimal (positive uphill)	,
`g2`	in	grade after the curve, decimal	,
`S`	in	required sight distance	m
`h1`	in	driver eye height (default 1.08 m) (optional)	m
`h2`	in	object height (default 0.60 m) (optional)	m
`L`	out	minimum curve length (m)	m
`case`	out	S < L, S > L or no curve needed	,

Applies when: g1 > g2; S > 0.

```
$ transportation solve transport_crest_curve_length "g1=0.03 1" "g2=-0.02 1" "S=185 m"
L = 260.070844 m
case = S < L
  by: Minimum length of a crest vertical curve for a sight distance  (S < L: L = A S^2 / (200
(sqrt(h1) + sqrt(h2))^2); S > L: L = 2S - 200 (sqrt(h1) + sqrt(h2))^2 / A (A in %))
  g1 = 0.03
  g2 = -0.02
  S = 185 m
  assumes: g1 > g2; S > 0
```

In the worked example, g1 = 0.03 1, g2 = -0.02 1, S = 185 m go through the equation above and give L = 260.070844 m, case = S < L. It applies only when g1 > g2; S > 0: outside that it says what is wrong instead of returning a number.

Minimum radius of a horizontal curve, `transport_minimum_curve_radius`

The smallest curve radius for a design speed v, maximum superelevation e and side friction factor f.

$$R_{min} = \frac{v^2(1 - ef)}{g(e + f)}$$

symbol		meaning	SI unit
`v`	in	design speed	m/s
`e`	in	superelevation rate, decimal	,
`f`	in	side friction factor	,
`R_min`	out	minimum curve radius (m)	m

Applies when: v > 0; f > 0; e f < 1.

```
$ transportation solve transport_minimum_curve_radius "v=27.77777778 m/s" "e=0.08 1" "f=0.12 1"
R_min = 389.4992513 m
  by: Minimum radius of a horizontal curve  (R = v^2 (1 - e f) / (g (e + f)))
  v = 27.77777778 m/s
  e = 0.08
  f = 0.12
  assumes: v > 0; f > 0; e f < 1
```

What the worked example means: This calculation finds the smallest safe horizontal curve radius for a given design speed, maximum superelevation, and side friction factor. Using a design speed of 27.7778 m/s, a superelevation rate of 0.08, and a side friction factor of 0.12, the resulting minimum radius is 389.499 m. This value ensures that vehicles can navigate the turn without exceeding physical limits of grip or banking, which is critical for meeting highway geometric design standards.

Passing sight distance (four-component method), `transport_passing_sight_distance`

The passing sight distance on a two-lane road as d1 + d2 + d3 + d4: the initial manoeuvre d1 = t1 (v - m + a t1/2), the time in the left lane d2 = v t2, the clearance d3 and the opposing vehicle's travel d4 = 2 d2/3 (AASHTO 2001).

$$PSD = t_1\left(v - m + \frac{at_1}{2}\right) + vt_2 + d_3 + \frac{2}{3}vt_2$$

symbol		meaning	SI unit
`v`	in	average speed of the passing vehicle	m/s
`m_diff`	in	speed difference between passing and passed vehicles	m/s
`a`	in	average acceleration during the initial manoeuvre	m/s^2
`t1`	in	time of the initial manoeuvre	s
`t2`	in	time in the left lane	s
`d3`	in	clearance between the passing and opposing vehicles	m
`PSD`	out	passing sight distance (m)	m
`d1`	out	initial manoeuvre distance (m)	m
`d2`	out	distance in the left lane (m)	m
`d3`	out	clearance (m)	m
`d4`	out	opposing vehicle distance (m)	m

Applies when: 0 < m < v; a, t1, t2 > 0; d3 >= 0; AASHTO 2001 four-component model.

```
$ transportation solve transport_passing_sight_distance "v=25 m/s" "m_diff=4.166666667 m/s" "a=0.64
m/s^2" "t1=4.3 s" "t2=10.4 s" "d3=60 m"
PSD = 588.8334667 m
d1 = 95.50013333 m
d2 = 260 m
d3 = 60 m
d4 = 173.3333333 m
  by: Passing sight distance (four-component method)  (d1 = t1 (v - m + a t1/2), d2 = v t2, d4 = 2
d2/3, PSD = d1 + d2 + d3 + d4)
  v = 25 m/s
  m_diff = 4.166666667 m/s
  a = 0.64 m/s^2
  t1 = 4.3 s
  t2 = 10.4 s
  d3 = 60 m
  assumes: 0 < m < v; a, t1, t2 > 0; d3 >= 0; AASHTO 2001 four-component model
```

What the worked example means: This calculation determines the minimum visible distance required for a driver to safely pass another vehicle on a two-lane road. It sums four distinct phases: the initial manoeuvre distance, which depends on the passing speed, speed difference, acceleration, and time; the distance traveled while in the left lane; the necessary clearance between vehicles; and the distance the opposing vehicle travels during that time. The result, 588.833 m, represents the total sight distance needed to complete the maneuver without collision.

Sag vertical curve length for headlight sight distance, `transport_sag_curve_headlight`

The minimum length of a sag vertical curve for the headlight sight distance S by the AASHTO formulas (headlight 0.6 m high, beam rising 1 degree): L = A S^2/(120 + 3.5 S) when S < L, L = 2S - (120 + 3.5 S)/A when S > L, A = g2 - g1 in percent.

$$L = \frac{AS^2}{120 + 3.5S}\ (S < L), \quad L = 2S - \frac{120 + 3.5S}{A}\ (S > L)$$

symbol		meaning	SI unit
`g1`	in	entering grade, decimal	,
`g2`	in	leaving grade, decimal	,
`S`	in	headlight sight distance	m
`L`	out	minimum curve length (m)	m
`A`	out	algebraic grade difference (%)	,
`K`	out	rate of vertical curvature L/A (m per %)	,
`case`	out	S < L or S > L	,

Applies when: g2 > g1 (a sag); AASHTO headlight height 0.6 m and 1 degree upward beam (coefficients 120 and 3.5).

```
$ transportation solve transport_sag_curve_headlight "g1=-0.03 1" "g2=0.02 1" "S=185 m"
L = 222.9641694 m
A = 5
K = 44.59283388
case = S < L
  by: Sag vertical curve length for headlight sight distance  (S < L: L = A S^2/(120 + 3.5 S); S >
L: L = 2S - (120 + 3.5 S)/A)
  g1 = -0.03
  g2 = 0.02
  S = 185 m
  assumes: g2 > g1 (a sag); AASHTO headlight height 0.6 m and 1 degree upward beam (coefficients
120 and 3.5)
```

What the worked example means: This calculation determines the shortest parabolic sag vertical curve required to ensure headlights illuminate the road surface up to the specified sight distance. Using an algebraic grade difference of 5 percent and a sight distance of 185 m, the resulting minimum curve length is 222.964 m. This ensures that drivers can see obstacles in low-light conditions based on standard headlight geometry.

Side friction demanded on a superelevated curve, `transport_side_friction_demand`

The side friction factor a vehicle needs at speed v on a curve of radius R with superelevation e (negative: the superelevation alone exceeds the demand and friction acts up the slope).

$$f = \frac{v^2/(gR) - e}{1 + e\,v^2/(gR)}$$

symbol		meaning	SI unit
`v`	in	design speed	m/s
`R`	in	curve radius	m
`e`	in	superelevation rate, decimal	,
`f`	out	side friction factor demanded	,

Applies when: v > 0; R > 0.

```
$ transportation solve transport_side_friction_demand "v=25 m/s" "R=250 m" "e=0.06 1"
f = 0.1919076305
  by: Side friction demanded on a superelevated curve  (f = (v^2/(gR) - e) / (1 + e v^2/(gR)))
  v = 25 m/s
  R = 250 m
  e = 0.06
  assumes: v > 0; R > 0
```

What the worked example means: This calculation determines the side friction factor required for a vehicle traveling at speed 25 m/s on a curve with radius 250 m and superelevation rate 0.06. The result, 0.191908, represents the lateral force per unit weight that tires must generate to maintain the path. A positive value indicates friction acts down the slope, while a negative value suggests the superelevation alone exceeds the demand, requiring friction to act up the slope.

Sight distance available on a curve with a roadside obstruction, `transport_sight_distance_from_offset`

The sight distance available on a curve of radius R (inside lane centreline) when an obstruction stands at the offset M from that centreline.

$$S = 2R \arccos\left(1 - \frac{M}{R}\right)$$

symbol		meaning	SI unit
`R`	in	radius of the inside lane centreline	m
`M`	in	offset to the obstruction	m
`S`	out	available sight distance (m)	m

Applies when: 0 < M < R.

```
$ transportation solve transport_sight_distance_from_offset "R=300 m" "M=9 m"
S = 147.3393105 m
  by: Sight distance available on a curve with a roadside obstruction  (S = 2R acos(1 - M / R))
  R = 300 m
  M = 9 m
  assumes: 0 < M < R
```

What the worked example means: This calculation determines the maximum sight distance a driver can achieve on a curved road section when a fixed object blocks part of the view. It uses the radius of the inside lane centreline, recorded as 300 m, and the perpendicular distance from that centreline to the obstruction, given as 9 m. The resulting value, 147.339 m, represents the length of road ahead that remains visible to the driver before being obscured by the object.

Sight distance available over a crest vertical curve, `transport_crest_curve_sight_distance`

The sight distance a crest vertical curve of length L between grades g1 and g2 provides (eye height 1.08 m, object height 0.60 m unless stated).

$$S = \sqrt{\frac{200\left(\sqrt{h_1} + \sqrt{h_2}\right)^2 L}{A}}$$

symbol		meaning	SI unit
`g1`	in	grade before the curve, decimal (positive uphill)	,
`g2`	in	grade after the curve, decimal	,
`L`	in	curve length	m
`h1`	in	driver eye height (default 1.08 m) (optional)	m
`h2`	in	object height (default 0.60 m) (optional)	m
`S`	out	available sight distance (m)	m
`case`	out	S < L or S > L	,

Applies when: g1 > g2; L >= 0.

```
$ transportation solve transport_crest_curve_sight_distance "g1=0.03 1" "g2=-0.02 1" "L=300 m"
S = 198.6948095 m
case = S < L
  by: Sight distance available over a crest vertical curve  (S = sqrt(200 (sqrt(h1) + sqrt(h2))^2 L
/ A) if that is <= L, else S = L/2 + 100 (sqrt(h1) + sqrt(h2))^2 / A)
  g1 = 0.03
  g2 = -0.02
  L = 300 m
  assumes: g1 > g2; L >= 0
```

What the worked example means: The available sight distance is 198.695 m m, determined by the curve length 300 m m and algebraic difference in grades.

Superelevation required on a horizontal curve, `transport_superelevation_required`

The superelevation rate needed at speed v on a curve of radius R when the side friction factor f is used.

$$e = \frac{v^2/(gR) - f}{1 + f\,v^2/(gR)}$$

symbol		meaning	SI unit
`v`	in	design speed	m/s
`R`	in	curve radius	m
`f`	in	side friction factor	,
`e`	out	superelevation rate (decimal)	,
`e_percent`	out	superelevation rate (%)	,

Applies when: v > 0; R > 0; f >= 0.

```
$ transportation solve transport_superelevation_required "v=27.77777778 m/s" "R=450 m" "f=0.12 1"
e = 0.05366318384
e_percent = 5.366318384
  by: Superelevation required on a horizontal curve  (e = (v^2/(gR) - f) / (1 + f v^2/(gR)))
  v = 27.77777778 m/s
  R = 450 m
  f = 0.12
  assumes: v > 0; R > 0; f >= 0
```

What the worked example means: This calculation determines the cross slope needed to safely navigate a horizontal curve. It uses the design speed of 27.7778 m/s and the curve radius of 450 m, while accounting for the available side friction factor of 0.12. The resulting superelevation rate is 0.0536632, which corresponds to a percentage grade of 5.36632. This value represents the banking angle that allows vehicles to maintain stability without relying solely on tire grip.

Mode choice (5)

Direct and cross elasticities of logit choice probabilities, `transport_logit_elasticities`

Point elasticities of logit probabilities with respect to an attribute x (such as cost or time) that enters the utility linearly with coefficient beta: direct E_ii = beta x_i (1 - P_i) and cross E_ji = -beta x_i P_i (the same for every other alternative j, the IIA property).

$$E_{x_i}^{P_i} = \beta x_i (1 - P_i), \quad E_{x_i}^{P_j} = -\beta x_i P_i$$

symbol		meaning	SI unit
`V`	in	utility of each alternative	,
`x`	in	value of the attribute for each alternative	,
`beta`	in	coefficient of the attribute	,
`P`	out	probability of each alternative	,
`direct`	out	direct elasticity of each alternative	,
`cross`	out	cross elasticity of the other alternatives with respect to each alternative's attribute	,

Applies when: one attribute value per alternative.

```
$ transportation solve transport_logit_elasticities "V=[-1.2, -0.8, -2.1]" "x=[30, 45, 20]"
beta=-0.04
P = [0.345018612617,0.514707287448,0.140274099936]
direct = [-0.78597766486,-0.873526882594,-0.687780720051]
cross = [0.41402233514,0.926473117406,0.112219279949]
  by: Direct and cross elasticities of logit choice probabilities  (E_ii = beta x_i (1 - P_i), E_ji
= -beta x_i P_i)
  beta = -0.04
  assumes: one attribute value per alternative
```

What the worked example means: This calculation determines how sensitive choice probabilities are to changes in an attribute like cost or time. The direct elasticity, shown as [-0.785978, -0.873527, -0.687781], measures the percentage change in the probability of choosing a specific alternative when its own attribute value changes. The cross elasticity, shown as [0.414022, 0.926473, 0.112219], measures the effect on all other

alternatives. In this logit model, the cross elasticity is uniform for every other option, reflecting the Independence of Irrelevant Alternatives property.

Logit mode choice from attributes and coefficients, `transport_logit_from_attributes`

Utilities V_i = ASC_i + sum_k beta_k x_ik of each mode from its attributes (rows of X: modes; columns: attributes such as time in min and cost) and the coefficients, and the resulting logit choice probabilities.

$$V_i = ASC_i + \sum_k \beta_k x_{ik}, \quad P_i = \frac{e^{V_i}}{\sum_j e^{V_j}}$$

symbol		meaning	SI unit
`X`	in	attributes: one row per mode, one column per attribute	,
`beta`	in	coefficient of each attribute	,
`asc`	in	alternative-specific constant of each mode (default 0) (optional)	,
`V`	out	utility of each mode	,
`P`	out	probability of each mode	,

Applies when: one coefficient per attribute column; at least two modes.

```
$ transportation solve transport_logit_from_attributes "X=[[30, 2.5], [45, 1.5], [60, 0.8]]" "beta=
[-0.05, -0.4]" "asc=[0, -0.5, -1]"
V = [-2.5,-3.35,-4.32]
P = [0.629152135572,0.268909017211,0.101938847218]
  by: Logit mode choice from attributes and coefficients  (V = X beta + ASC, P = softmax(V))
  assumes: one coefficient per attribute column; at least two modes
```

What the worked example means: This calculation determines the utility for each transportation mode by combining its specific attributes with their corresponding coefficients, plus any alternative-specific constants. The input matrix of attributes [30 2.5; 45 1.5; 60 0.8] is multiplied by the coefficient vector [-0.05, -0.4], and these results are added to the constants [0, -0.5, -1] to produce the utility values [-2.5, -3.35, -4.32]. These utilities are then transformed into probabilities [0.629152, 0.268909, 0.101939] using a softmax function, which represents the likelihood that a traveler will choose each available mode based on the estimated preferences.

Multinomial logit mode shares from utilities, `transport_logit_mode_shares`

Choice probabilities (mode shares) of a multinomial logit model from the systematic utilities of the alternatives, and the logsum (expected maximum utility).

$$P_i = \frac{e^{V_i}}{\sum_j e^{V_j}}$$

symbol		meaning	SI unit
`V`	in	utility of each alternative	,
`P`	out	probability of each alternative	,
`logsum`	out	logsum ln(sum exp V)	,

Applies when: at least two alternatives.

```
$ transportation solve transport_logit_mode_shares "V=[-1.2, -0.8, -2.1]"
P = [0.345018612617,0.514707287448,0.140274099936]
logsum = -0.1358430862
  by: Multinomial logit mode shares from utilities  (P_i = exp(V_i) / sum_j exp(V_j))
  assumes: at least two alternatives
```

What the worked example means: This calculation determines the probability of choosing each transport mode based solely on the utility scores of those modes. The resulting probabilities, represented as [0.345019, 0.514707, 0.140274], indicate the share of travelers expected to select each alternative given their systematic utilities, denoted here as [-1.2, -0.8, -2.1]. Additionally, the logsum value, -0.135843, provides a measure of the expected maximum utility across all available options, serving as a key metric for evaluating overall system attractiveness.

Nested logit choice probabilities, `transport_nested_logit`

Choice probabilities of a two-level nested logit model (for example car / public transport nests holding bus and rail): alternatives grouped into nests 1..K with dissimilarity (logsum) parameters lam_k in (0, 1]; also the probability of each nest.

$$P_i = \frac{e^{V_i/\lambda_k}}{e^{I_k}} \cdot \frac{e^{\lambda_k I_k}}{\sum_m e^{\lambda_m I_m}}, \quad I_k = \ln \sum_{j \in k} e^{V_j/\lambda_k}$$

symbol		meaning	SI unit
`V`	in	utility of each alternative	,
`nest`	in	nest number (1, 2, ...) of each alternative	,
`lam`	in	dissimilarity parameter of each nest, in (0, 1]	,
`P`	out	probability of each alternative	,
`P_nest`	out	probability of each nest	,

Applies when: nests numbered 1..K, each used; 0 < lam_k <= 1.

```
$ transportation solve transport_nested_logit "V=[-0.5, -1.2, -1]" "nest=[1, 2, 2]" "lam=[1, 0.5]"
P = [0.560574109573,0.176347032294,0.263078858132]
P_nest = [0.560574109573,0.439425890427]
  by: Nested logit choice probabilities  (I_k = ln sum_{j in k} exp(V_j / lam_k); P(k) = exp(lam_k
I_k) / sum_m exp(lam_m I_m); P(i|k) = exp(V_i/lam_k - I_k))
  assumes: nests numbered 1..K, each used; 0 < lam_k <= 1
```

What the worked example means: This calculation determines the likelihood that a traveler selects a specific mode or route by applying a nested logit model. It uses the utility values [-0.5, -1.2, -1] and dissimilarity parameters [1, 0.5] to compute intermediate inclusive value measures, which are then combined to yield the final alternative probabilities [0.560574, 0.176347, 0.263079] and nest probabilities [0.560574, 0.439426]. The result reflects how alternatives within similar groups, such as different public transport options, share substitution patterns distinct from those between unrelated groups.

User benefit of a change from the logit logsum, `transport_logsum_user_benefit`

Change in consumer surplus per trip (in money units) when the utilities of the alternatives change from V0 to V1, from the difference of the logsums divided by the (negative) cost coefficient.

$$\Delta CS = \frac{1}{-\beta_c}\left[\ln\sum_j e^{V_j^1} - \ln\sum_j e^{V_j^0}\right]$$

symbol		meaning	SI unit
`V0`	in	utilities before the change	,
`V1`	in	utilities after the change	,
`beta_cost`	in	cost coefficient (utility per money unit, negative)	,
`dCS`	out	change in consumer surplus per trip (money units)	,
`logsum0`	out	logsum before	,
`logsum1`	out	logsum after	,

Applies when: same alternatives before and after; beta_cost < 0.

```
$ transportation solve transport_logsum_user_benefit "V0=[-1.2, -0.8, -2.1]" "V1=[-1.2, -0.5,
-1.6]" beta_cost=-0.25
dCS = 0.9594478117
logsum0 = -0.1358430862
logsum1 = 0.1040188667
  by: User benefit of a change from the logit logsum  (dCS = (ln sum exp V1 - ln sum exp V0) / (-
beta_cost))
  beta_cost = -0.25
  assumes: same alternatives before and after; beta_cost < 0
```

What the worked example means: This calculation determines the monetary value of the welfare change for a traveler when utilities shift from their initial state to a new one. It compares the logsum of utilities before the change, which is -0.135843, with the logsum after the change, valued at 0.104019. The difference between these two values is divided by the negative cost coefficient, represented here as -0.25, to convert utility differences into money units. The resulting figure, 0.959448, indicates how much better or worse off the average user is due to the policy or infrastructure change.

Queueing (5)

Blocking probability with no waiting room (Erlang B), e.g. parking spaces, `transport_erlang_b_blocking`

Arrivals at lam (veh/h) to c spaces or servers held for a mean time 1/mu with no waiting room (a car park, loading bays, taxi ranks): the probability an arrival is turned away (Erlang B), the mean number of occupied spaces and the vehicles lost per hour. Rates in veh/h.

$$B(c,a) = \frac{a^c/c!}{\sum_{k=0}^{c} a^k/k!}$$

symbol		meaning	SI unit
`lam`	in	arrival rate (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`mu`	in	service rate per server (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s

symbol		meaning	SI unit
`c`	in	number of spaces or servers (whole number)	,
`P_block`	out	probability an arrival is turned away	,
`carried`	out	mean number of occupied spaces	,
`lost_per_h`	out	vehicles turned away (veh/h)	1/h

Applies when: 1 <= c <= 1000.

```
$ transportation solve transport_erlang_b_blocking "lam=60 1/h" "mu=0.5 1/h" c=120
P_block = 0.06941876906
carried = 111.6697477
lost_per_h = 4.165126144 1/h
  by: Blocking probability with no waiting room (Erlang B), e.g. parking spaces  (Erlang B
recursion B(k) = a B(k-1) / (k + a B(k-1)), a = lam/mu)
  lam = 0.01666666667 1/s
  mu = 0.0001388888889 1/s
  assumes: 1 <= c <= 1000
```

What the worked example means: This calculation determines the likelihood that an arriving vehicle finds all parking spaces occupied and must leave without service. Given an arrival rate of 60 1/h vehicles per hour and a mean occupancy time derived from a service rate of 0.5 1/h, the system offers an offered load based on these rates. With 120 available spaces, the computed blocking probability is 0.0694188. Consequently, the average number of occupied spaces is 111.67, and the traffic loss amounts to 4.16513 1/h vehicles per hour.

First-come-first-served queue from arrival and service times, `transport_fifo_queue_simulation`

Waiting time of every vehicle (or passenger) at c identical servers in first-come-first-served order from given arrival times and service times (a toll plaza, a bus stop with c berths): mean and longest wait, mean time in the system, when the last one leaves and the longest waiting line. Times in s.

$$S_n = \max(A_n,\ \min_j F_j), \quad W_n = S_n - A_n$$

symbol		meaning	SI unit
`arrivals`	in	arrival times in s, increasing	,
`service`	in	service times in s (one per arrival, or one for all)	,
`c`	in	number of servers (default 1) (optional)	,
`waits`	out	waiting time of each arrival (s)	,
`mean_wait`	out	mean waiting time (s)	s
`max_wait`	out	longest wait (s)	s
`mean_time_in_system`	out	mean time in the system (s)	s
`last_departure`	out	time the last one leaves (s)	s
`max_queue`	out	longest waiting line	,

Applies when: arrival times increasing; service times > 0; c >= 1.

```
$ transportation solve transport_fifo_queue_simulation "arrivals=[0, 5, 7, 12, 14, 20, 21]"
"service=[8, 6, 9, 4, 7, 5, 6]"
waits = [0.0,3.0,7.0,11.0,13.0,14.0,18.0]
mean_wait = 9.428571429 s
max_wait = 18 s
mean_time_in_system = 15.85714286 s
last_departure = 45 s
max_queue = 4
  by: First-come-first-served queue from arrival and service times  (each arrival starts at
max(arrival, earliest free server) (Kiefer-Wolfowitz recursion))
  assumes: arrival times increasing; service times > 0; c >= 1
```

What the worked example means: This calculation determines the waiting times for each vehicle or passenger at a service point with identical servers, using first-come-first-served logic. By processing the arrival and service times, it computes that the average wait is 9.42857 s and the peak wait reached 18 s. The total time spent in the system averaged 15.8571 s, and the last individual departed at 45 s. The maximum number of entities queued simultaneously was 4.

Multi-server random queue (M/M/c, Erlang C), e.g. a toll plaza, `transport_mmc_queue`

Poisson arrivals at lam served by c parallel exponential servers at mu each (toll lanes, loading bays, ticket windows): probability of waiting (Erlang C), mean queue, mean wait, mean time in the system and mean number in the system. Rates in veh/h.

$$P_W = \frac{\frac{a^c}{c!}\frac{1}{1-\rho}}{\sum_{k=0}^{c-1}\frac{a^k}{k!} + \frac{a^c}{c!}\frac{1}{1-\rho}}, \quad L_q = P_W\frac{\rho}{1-\rho}$$

symbol		meaning	SI unit
`lam`	in	arrival rate (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`mu`	in	service rate per server (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`c`	in	number of servers (whole number)	,
`P_wait`	out	probability an arrival waits (Erlang C)	,
`Lq`	out	mean number waiting (veh)	,
`Wq`	out	mean waiting time (s)	s
`W`	out	mean time in the system (s)	s
`L`	out	mean number in the system (veh)	,

Applies when: lam < c mu; 1 <= c <= 200.

```
$ transportation solve transport_mmc_queue "lam=1500 1/h" "mu=400 1/h" c=5
P_wait = 0.461788959
Lq = 1.385366877
Wq = 3.324880505 s
W = 12.3248805 s
L = 5.135366877
  by: Multi-server random queue (M/M/c, Erlang C), e.g. a toll plaza  (Erlang C: P_wait =
(a^c/c!)/(1 - rho) / (sum_{k<c} a^k/k! + (a^c/c!)/(1 - rho)); Lq = P_wait rho/(1 - rho))
  lam = 0.4166666667 1/s
  mu = 0.1111111111 1/s
  assumes: lam < c mu; 1 <= c <= 200
```

What the worked example means: This calculation determines queue behavior for a multi-server system where vehicles arrive at a rate of 1500 1/h and are served by 5 parallel channels each processing at 400 1/h. The result shows that the probability an arriving vehicle must wait is 0.461789, leading to a mean queue length of 1.38537 vehicles. Consequently, the average time spent waiting in line is 3.32488 s seconds, while the total time in the system averages 12.3249 s seconds.

Single server with constant service time (M/D/1), `transport_md1_queue`

Poisson arrivals at lam and a constant service time 1/mu (an automated toll gate, a ramp meter): mean queue, mean wait, mean time in the system and mean number in the system. Rates in veh/h.

$$L_q = \frac{\rho^2}{2(1-\rho)}, \quad W_q = \frac{\rho}{2\mu(1-\rho)}$$

symbol		meaning	SI unit
`lam`	in	arrival rate (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`mu`	in	service rate per server (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`Lq`	out	mean number waiting (veh)	,
`Wq`	out	mean waiting time (s)	s
`W`	out	mean time in the system (s)	s
`L`	out	mean number in the system (veh)	,

Applies when: lam < mu.

```
$ transportation solve transport_md1_queue "lam=400 1/h" "mu=500 1/h"
Lq = 1.6
Wq = 14.4 s
W = 21.6 s
L = 2.4
  by: Single server with constant service time (M/D/1)  (Lq = rho^2 / (2(1 - rho)), Wq = rho / (2
mu (1 - rho)), W = (2 - rho)/(2 mu (1 - rho)))
  lam = 0.1111111111 1/s
  mu = 0.1388888889 1/s
  assumes: lam < mu
```

What the worked example means: This calculation determines the performance of a single-server system with constant service times, such as an automated toll gate. Given an arrival rate of 400 1/h and a service rate

of 500 1/h, the model computes that the average number of vehicles waiting in the queue is 1.6, while the average time each vehicle spends waiting before service begins is 14.4 s seconds. The total average time in the system, including service, is 21.6 s seconds, resulting in an average total occupancy of 2.4 vehicles.

Single-server random queue (M/M/1), e.g. a toll booth, `transport_mm1_queue`

Poisson arrivals at lam and exponential service at mu at one server (a toll booth, a ramp meter): utilisation, mean number in the system and in the queue, mean time in the system and waiting, and the probability that the server is idle. Rates in veh/h.

$$L = \frac{\rho}{1-\rho}, \quad W = \frac{1}{\mu - \lambda}, \quad W_q = \frac{\rho}{\mu - \lambda}$$

symbol		meaning	SI unit
`lam`	in	arrival rate (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`mu`	in	service rate per server (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`rho`	out	utilisation lam / mu	,
`L`	out	mean number in the system (veh)	,
`Lq`	out	mean number waiting (veh)	,
`W`	out	mean time in the system (s)	s
`Wq`	out	mean waiting time (s)	s
`P0`	out	probability the system is empty	,

Applies when: lam < mu.

```
$ transportation solve transport_mm1_queue "lam=400 1/h" "mu=500 1/h"
rho = 0.8
L = 4
Lq = 3.2
W = 36 s
Wq = 28.8 s
P0 = 0.2
  by: Single-server random queue (M/M/1), e.g. a toll booth  (rho = lam/mu, L = rho/(1 - rho), Lq =
rho^2/(1 - rho), W = 1/(mu - lam), Wq = rho/(mu - lam))
  lam = 0.1111111111 1/s
  mu = 0.1388888889 1/s
  assumes: lam < mu
```

What the worked example means: This calculation determines the steady-state performance of a single-server queue, such as a lone toll booth. Using the arrival rate of 400 1/h and service rate of 500 1/h, it computes that the server is busy 0.8 of the time. Consequently, there are typically 4 vehicles in the entire system, including those being served, while 3.2 are waiting in line. The average vehicle spends 36 s seconds in the system total, with 28.8 s seconds spent purely waiting for service.

Railway operations (4)

Minimum headway of trains at a station stop (moving block), `transport_rail_station_headway`

Shortest headway between trains serving a station under a brick-wall (moving-block) separation: the leader dwells t_d, accelerates at a until its rear clears the platform plus the safety margin s_m, while the follower approaches at the line speed v and brakes at b to stop at the platform; and the corresponding line capacity in trains per hour.

$$h = t_d + \sqrt{\frac{2(L_t + s_m)}{a}} + \frac{v}{b}$$

symbol		meaning	SI unit
`v`	in	line (approach) speed	m/s
`a`	in	acceleration	m/s^2
`b`	in	braking rate	m/s^2
`L_t`	in	train length	m
`t_d`	in	dwell time	s
`s_m`	in	safety margin (default 0) (optional)	m
`h`	out	minimum headway (s)	s
`trains_per_h`	out	line capacity (trains/h)	1/h

Applies when: v, a, b, L_t > 0; t_d, s_m >= 0.

```
$ transportation solve transport_rail_station_headway "v=22.22222222 m/s" "a=1 m/s^2" "b=1.1 m/s^2"
"L_t=150 m" "t_d=30 s" "s_m=50 m"
h = 70.2020202 s
trains_per_h = 51.28057554 1/h
  by: Minimum headway of trains at a station stop (moving block)  (h = t_d + t_clear + v/b, t_clear
= sqrt(2 (L + s)/a) (or v/(2a) + (L + s)/v once the line speed is reached))
  v = 22.22222222 m/s
  a = 1 m/s^2
  b = 1.1 m/s^2
  L_t = 150 m
  t_d = 30 s
  s_m = 50 m
  assumes: v, a, b, L_t > 0; t_d, s_m >= 0
```

What the worked example means: This calculation determines the shortest safe time interval between consecutive trains stopping at a station under moving block signaling. It combines the leading train's dwell time and acceleration phase to clear the platform with the following train's braking distance from line speed. The resulting headway of 70.202 s seconds directly dictates the maximum throughput, which is computed as 51.2806 1/h trains per hour.

Running time between stations (accelerate, cruise, brake), `transport_train_running_time`

Minimum running time of a train (or bus) between two stops a distance D apart with acceleration a, braking rate b and line speed v_max; when the stops are too close to reach v_max the profile is accelerate-brake.

$$T = \frac{D}{v} + \frac{v}{2a} + \frac{v}{2b}$$

symbol		meaning	SI unit
`D`	in	distance between stops	m
`v_max`	in	maximum (line) speed	m/s
`a`	in	acceleration	m/s^2
`b`	in	braking rate	m/s^2
`T`	out	running time stop to stop (s)	s
`v_peak`	out	highest speed reached (m/s)	m/s
`profile`	out	accelerate-cruise-brake or accelerate-brake	,

Applies when: all inputs positive.

```
$ transportation solve transport_train_running_time "D=1500 m" "v_max=22.22222222 m/s" "a=1 m/s^2"
"b=1.2 m/s^2"
T = 87.87037038 s
v_peak = 22.22222222 m/s
profile = accelerate-cruise-brake
  by: Running time between stations (accelerate, cruise, brake)  (if v^2/(2a) + v^2/(2b) <= D: T =
D/v + v/(2a) + v/(2b); else v_p = sqrt(2 D a b / (a + b)), T = v_p/a + v_p/b)
  D = 1500 m
  v_max = 22.22222222 m/s
  a = 1 m/s^2
  b = 1.2 m/s^2
  assumes: all inputs positive
```

What the worked example means: This calculation determines the minimum time for a vehicle to travel between two stops, given the distance 1500 m, maximum line speed 22.2222 m/s, acceleration 1 m/s^2, and braking rate 1.2 m/s^2. The result shows a total running time of 87.8704 s, reaching a highest speed of 22.2222 m/s. Depending on whether the distance allows the vehicle to reach its top speed before braking, the motion follows either an accelerate-cruise-brake or an accelerate-brake profile, indicated here as accelerate-cruise-brake.

Train acceleration time and distance (tractive effort curve, Davis resistance), `transport_train_acceleration`

Time and distance for a train to accelerate from v1 to v2 with a tractive effort F_max up to the speed P / F_max and constant power P above it, against the Davis resistance A + B v + C v^2 and the gradient, with a rotating-mass factor gamma.

$$t = \int_{v_1}^{v_2} \frac{\gamma m \, dv}{\min(F_{max}, P/v) - (A + Bv + Cv^2) - mg\sin\theta}$$

symbol		meaning	SI unit
`m`	in	train mass	kg
`v1`	in	initial speed	m/s
`v2`	in	final speed	m/s
`F_max`	in	maximum (starting) tractive effort	N
`P`	in	power at the wheel rim	W

symbol		meaning	SI unit
`A`	in	Davis coefficient A (constant resistance)	N
`B`	in	Davis coefficient B (per speed)	N*s/m
`C`	in	Davis coefficient C (per speed squared)	N*s^2/m^2
`G`	in	grade, decimal (positive uphill) (optional)	,
`gamma`	in	rotating-mass factor (default 1.08) (optional)	,
`t`	out	acceleration time (s)	s
`d`	out	distance travelled (m)	m

Applies when: tractive effort exceeds resistance up to v2; v2 > v1 >= 0.

```
$ transportation solve transport_train_acceleration "m=400000 kg" "v1=0 m/s" "v2=27.77777778 m/s"
"F_max=150000 N" "P=4000000 W" "A=6000 N" "B=180 N*s/m" "C=10.368 N*s^2/m^2"
t = 86.5875385 s
d = 1222.691944 m
  by: Train acceleration time and distance (tractive effort curve, Davis resistance)  (t = integral
of gamma m dv / (F(v) - R(v) - m g sin(theta)), F(v) = min(F_max, P/v) (quadrature))
  m = 400000 kg
  v1 = 0 m/s
  v2 = 27.77777778 m/s
  F_max = 150000 N
  P = 4000000 W
  A = 6000 N
  B = 180 N*s/m
  C = 10.368 N*s^2/m^2
  assumes: tractive effort exceeds resistance up to v2; v2 > v1 >= 0
```

What the worked example means: This calculation determines the time and distance required for a train to accelerate between two specific speeds. It considers the engine's tractive effort limits, power constraints, rolling resistance, aerodynamic drag, gradient effects, and rotational inertia. The result shows that under these conditions, the acceleration process lasts 86.5875 s seconds over a path length of 1222.69 m meters.

Train braking distance with Davis resistance and gradient, `transport_train_braking_distance`

Braking distance and time of a train of mass m braking with a constant brake force F_b from v1 to v2, helped by the Davis train resistance R = A + B v + C v^2 and the gradient (positive uphill), with a rotating-mass factor gamma.

$$S = \int_{v_2}^{v_1} \frac{\gamma m \, v \, dv}{F_b + A + Bv + Cv^2 + mg\sin\theta}$$

symbol		meaning	SI unit
`m`	in	train mass	kg
`v1`	in	initial speed	m/s
`F_b`	in	brake force	N
`A`	in	Davis coefficient A (constant resistance)	N
`B`	in	Davis coefficient B (per speed)	N*s/m

symbol		meaning	SI unit
`C`	in	Davis coefficient C (per speed squared)	N*s^2/m^2
`G`	in	grade, decimal (positive uphill) (optional)	,
`gamma`	in	rotating-mass factor (default 1.06) (optional)	,
`v2`	in	final speed (default 0) (optional)	m/s
`S`	out	braking distance (m)	m
`t`	out	braking time (s)	s

Applies when: F_b + R(v) + grade force > 0 over the speed range; v2 < v1.

```
$ transportation solve transport_train_braking_distance "m=400000 kg" "v1=33.33333333 m/s"
"F_b=320000 N" "A=6000 N" "B=180 N*s/m" "C=10.368 N*s^2/m^2"
S = 701.698286 m
t = 42.47286839 s
  by: Train braking distance with Davis resistance and gradient  (S = integral from v2 to v1 of
gamma m v dv / (F_b + A + B v + C v^2 + m g sin(theta)) (quadrature))
  m = 400000 kg
  v1 = 33.33333333 m/s
  F_b = 320000 N
  A = 6000 N
  B = 180 N*s/m
  C = 10.368 N*s^2/m^2
  assumes: F_b + R(v) + grade force > 0 over the speed range; v2 < v1
```

What the worked example means: The train requires 701.698 m to stop from 33.3333 m/s, taking 42.4729 s.

Route planning (2)

Fastest route on a road network (link lengths and speeds), `transport_fastest_route`

The quickest route between two places on a road network whose links are given by length (km) and speed (km/h): the route, its travel time in minutes and its length in km. Roads are two-way unless directed = yes.

$$t_{link} = \frac{60\,L}{v}, \quad \min \sum t_{link}$$

symbol		meaning	SI unit
`roads`	in	roads as 'A-B length' with lengths in km	,
`speeds`	in	speed on each road in km/h, in the same order	,
`origin`	in	starting place	,
`destination`	in	destination	,
`directed`	in	one-way links as written (default no) (optional)	,
`route`	out	fastest route (places joined by '-')	,
`time`	out	travel time (min)	min
`distance`	out	route length (km)	km

Applies when: one positive length and speed per road; the destination is reachable.

```
$ transportation solve transport_fastest_route "roads=A-B 4, A-C 2.5, B-C 1.2, B-D 5, C-D 8, C-E
10, D-E 2" "speeds=60, 50, 40, 80, 100, 90, 50" origin=A destination=E
route = A-C-E
time = 9.666666667 min
distance = 12.5 km
  by: Fastest route on a road network (link lengths and speeds)  (Dijkstra's algorithm on link
travel times 60 L / v (min))
  origin = A
  destination = E
  assumes: one positive length and speed per road; the destination is reachable
```

What the worked example means: This calculation determines the quickest path between two locations on a road network by minimizing total travel time. It uses the length and speed of each road segment to compute individual link times, then sums these for all possible paths to find the minimum. The result shows that traveling via A-C-E takes 9.66667 min minutes and covers a distance of 12.5 km kilometers, representing the optimal choice under free-flow conditions.

Zone-to-zone travel time matrix (skim) of a road network, `transport_travel_time_skim`

The fastest travel time in minutes between every pair of places of a road network given by link lengths (km) and speeds (km/h), as a matrix in alphabetical order of the places (the skim used for trip distribution).

$$t_{ij} = \min_{paths\ i \to j} \sum t_{link}$$

symbol		meaning	SI unit
`roads`	in	roads as 'A-B length' with lengths in km	,
`speeds`	in	speed on each road in km/h, in the same order	,
`directed`	in	one-way links as written (default no) (optional)	,
`places`	out	places in the order of the matrix	,
`times`	out	travel time matrix (min)	,

Applies when: every place reachable; at most 60 places.

```
$ transportation solve transport_travel_time_skim "roads=A-B 4, A-C 2.5, B-C 1.2, B-D 5, C-D 8, C-E
10, D-E 2" "speeds=60, 50, 40, 80, 100, 90, 50"
places = A, B, C, D, E
times = [[0.0,4.0,3.0,7.75,9.66666666667],[4.0,0.0,1.8,3.75,6.15],[3.0,1.8,0.0,4.8,6.66666666667],
[7.75,3.75,4.8,0.0,2.4],[9.66666666667,6.15,6.66666666667,2.4,0.0]]
  by: Zone-to-zone travel time matrix (skim) of a road network  (Dijkstra from every place)
  assumes: every place reachable; at most 60 places
```

What the worked example means: This calculation produces a zone-to-zone travel time matrix by applying Dijkstra's algorithm to every node in the road network. It uses link lengths provided in A-B (4), A-C (2.5), B-C (1.2), B-D (5), C-D (8), C-E (10), D-E (2) and speeds in [60, 50, 40, 80, 100, 90, 50] to determine the fastest route between each pair of locations. The resulting matrix, labeled with places in alphabetical order such as A, B, C, D, E, contains the minimum travel times in [[0, 4, 3, 7.75, 9.66667], [4, 0, 1.8, 3.75, 6.15], [3, 1.8, 0, 4.8, 6.66667], [7.75, 3.75, 4.8, 0, 2.4], [9.66667, 6.15, 6.66667, 2.4, 0]] minutes, which serves as essential input for trip distribution models.

Traffic flow (14)

Fit the Greenberg (logarithmic) model to speed-density observations, `transport_greenberg_fit`

Least-squares fit of Greenberg's model v = v0 ln(kj / k) (v against ln k) to observed speeds (km/h) and densities (veh/km): the optimum speed v0, jam density, capacity v0 kj / e and the coefficient of determination.

$$v = v_0 \ln \frac{k_j}{k}, \quad q_{max} = \frac{v_0 k_j}{e}$$

symbol		meaning	SI unit
`speeds`	in	observed space-mean speeds in km/h	,
`densities`	in	observed densities in veh/km, in the same order	,
`v0`	out	optimum speed v0 (km/h)	km/h
`kj`	out	jam density (veh/km)	1/km
`q_max`	out	capacity (veh/h)	1/h
`r2`	out	coefficient of determination (v on ln k)	,

Applies when: at least three observations; speeds fall with density.

```
$ transportation solve transport_greenberg_fit "speeds=[82, 70, 58, 47, 33, 21]" "densities=[18,
32, 46, 60, 78, 95]"
v0 = 36.43837575 km/h
kj = 197.8362218 1/km
q_max = 2651.980568 1/h
r2 = 0.9573293444
  by: Fit the Greenberg (logarithmic) model to speed-density observations  (least squares of v on
ln k: slope -v0, intercept v0 ln kj; q_max = v0 kj / e)
  assumes: at least three observations; speeds fall with density
```

What the worked example means: This calculation applies the least squares method to regress observed speeds against the natural logarithm of densities. The process determines the Greenberg model parameters, specifically the optimum speed 36.4384 km/h and the jam density 197.836 1/km. These values define the theoretical maximum flow rate, or capacity, which is computed as 2651.98 1/h. The coefficient of determination 0.957329 indicates how well the logarithmic curve fits the provided data points.

Fit the Greenshields model to speed-density observations, `transport_greenshields_fit`

Least-squares fit of Greenshields' line v = vf - (vf/kj) k to observed speeds (km/h) and densities (veh/km): the free-flow speed, jam density, capacity and the coefficient of determination of the line.

$$v = v_f - \frac{v_f}{k_j} k, \quad q_{max} = \frac{v_f k_j}{4}$$

symbol		meaning	SI unit
`speeds`	in	observed space-mean speeds in km/h	,
`densities`	in	observed densities in veh/km, in the same order	,

symbol		meaning	SI unit
`vf`	out	free-flow speed (km/h)	km/h
`kj`	out	jam density (veh/km)	1/km
`q_max`	out	capacity (veh/h)	1/h
`r2`	out	coefficient of determination of the fit	,

Applies when: at least three observations; speeds fall with density.

```
$ transportation solve transport_greenshields_fit "speeds=[82, 70, 58, 47, 33, 21]" "densities=[18,
32, 46, 60, 78, 95]"
vf = 95.35371214 km/h
kj = 120.1405416 1/km
q_max = 2863.961656 1/h
r2 = 0.9986899158
  by: Fit the Greenshields model to speed-density observations  (least squares of v on k: intercept
vf, slope -vf/kj; q_max = vf kj / 4)
  assumes: at least three observations; speeds fall with density
```

What the worked example means: This calculation performs a least-squares regression on the observed space-mean speeds and densities to estimate the parameters of the Greenshields model. The resulting free-flow speed is 95.3537 km/h km/h, representing the theoretical maximum speed at zero density. The estimated jam density is 120.141 1/km veh/km, indicating the density at which traffic comes to a complete stop. Based on these linear parameters, the derived capacity, or maximum flow rate, is 2863.96 1/h veh/h. The coefficient of determination for this linear fit is 0.99869, which quantifies how well the straight-line model explains the variation in the observed speed-density data.

Fit the Underwood (exponential) model to speed-density observations, `transport_underwood_fit`

Least-squares fit of Underwood's model v = vf exp(-k / k0) (ln v against k) to observed speeds (km/h) and densities (veh/km): the free-flow speed, the optimum density k0, capacity vf k0 / e and the coefficient of determination.

$$v = v_f\, e^{-k/k_0}, \quad q_{max} = \frac{v_f k_0}{e}$$

symbol		meaning	SI unit
`speeds`	in	observed space-mean speeds in km/h	,
`densities`	in	observed densities in veh/km, in the same order	,
`vf`	out	free-flow speed (km/h)	km/h
`k0`	out	optimum density (veh/km)	1/km
`q_max`	out	capacity (veh/h)	1/h
`r2`	out	coefficient of determination (ln v on k)	,

Applies when: at least three observations; speeds fall with density.

```
$ transportation solve transport_underwood_fit "speeds=[82, 70, 58, 47, 33, 21]" "densities=[18,
32, 46, 60, 78, 95]"
vf = 122.3015297 km/h
k0 = 57.3960457 1/km
q_max = 2582.375424 1/h
r2 = 0.9749558489
  by: Fit the Underwood (exponential) model to speed-density observations  (least squares of ln v
on k: intercept ln vf, slope -1/k0; q_max = vf k0 / e)
  assumes: at least three observations; speeds fall with density
```

What the worked example means: This calculation determines the parameters of the Underwood model by performing a least-squares regression on the natural logarithm of observed speeds against densities. The intercept of this linear fit provides the free-flow speed, estimated here as 122.302 km/h, while the negative slope corresponds to the inverse of the optimum density, which is calculated as 57.396 1/km. Using these parameters, the theoretical maximum flow or capacity is derived as 2582.38 1/h. The quality of the exponential fit is indicated by the coefficient of determination, reported as 0.974956.

Greenberg model: capacity and density at capacity, `transport_greenberg_capacity`

Capacity of a traffic stream following Greenberg's model v = v0 ln(kj / k) and the density at which it occurs (the speed there is v0).

$$q_{max} = \frac{v_0 k_j}{e}$$

symbol		meaning	SI unit
`v0`	in	optimum speed v0	m/s
`kj`	in	jam density (veh/km in questions; veh/m in solve(), or '25 1/km')	1/m
`q_max`	out	capacity (veh/h)	1/h
`k_m`	out	density at capacity (veh/km)	1/km

Applies when: v0 > 0; kj > 0.

```
$ transportation solve transport_greenberg_capacity "v0=8.333333333 m/s" "kj=140 1/km"
q_max = 1545.093653 1/h
k_m = 51.50312176 1/km
  by: Greenberg model: capacity and density at capacity  (q = v0 k ln(kj/k) is largest at k = kj /
e: q_max = v0 kj / e)
  v0 = 8.333333333 m/s
  kj = 0.14 1/m
  assumes: v0 > 0; kj > 0
```

What the worked example means: This calculation determines the peak traffic throughput for a stream following Greenberg's logarithmic speed-density model. Using an optimum speed parameter of 8.33333 m/s and a jam density of 140 1/km, the system yields a maximum capacity of 1545.09 1/h. This highest possible flow rate occurs when the traffic density reaches 51.5031 1/km, which corresponds to the point where the derivative of the flow equation equals zero.

Greenshields model: capacity, optimum speed and density, `transport_greenshields_capacity`

Capacity (maximum flow) of a traffic stream following Greenshields' linear speed-density model v = vf (1 - k/kj), with the density and speed at which it occurs.

$$q_{max} = \frac{v_f k_j}{4}, \quad k_m = \frac{k_j}{2}, \quad v_m = \frac{v_f}{2}$$

symbol		meaning	SI unit
`vf`	in	free-flow speed	m/s
`kj`	in	jam density (veh/km in questions; veh/m in solve(), or '25 1/km')	1/m
`q_max`	out	capacity (veh/h)	1/h
`k_m`	out	density at capacity (veh/km)	1/km
`v_m`	out	speed at capacity (km/h)	km/h

Applies when: vf > 0; kj > 0.

```
$ transportation solve transport_greenshields_capacity "vf=27.77777778 m/s" "kj=120 1/km"
q_max = 3000 1/h
k_m = 60 1/km
v_m = 50 km/h
  by: Greenshields model: capacity, optimum speed and density  (q_max = vf kj / 4 at k = kj/2, v =
vf/2)
  vf = 27.77777778 m/s
  kj = 0.12 1/m
  assumes: vf > 0; kj > 0
```

What the worked example means: This calculation determines the maximum flow rate, or capacity, of a traffic stream based on the Greenshields linear speed-density model. Using the free-flow speed of 27.7778 m/s and the jam density of 120 1/km, the model predicts that the peak flow reaches 3000 1/h. This maximum occurs when the vehicle density is half of the jam density, calculated as 60 1/km, and the average speed is half of the free-flow speed, calculated as 50 km/h. These values represent the theoretical throughput limit of the roadway under idealized homogeneous traffic conditions.

Greenshields model: densities and speeds at a given flow, `transport_greenshields_speed_at_flow`

The two traffic states (uncongested and congested) of a Greenshields stream that carry the flow q: their densities and speeds.

$$k = \frac{k_j}{2}\left(1 \mp \sqrt{1 - \frac{4q}{v_f k_j}}\right)$$

symbol		meaning	SI unit
`vf`	in	free-flow speed	m/s
`kj`	in	jam density (veh/km in questions; veh/m in solve(), or '25 1/km')	1/m
`q`	in	traffic flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`k_1`	out	uncongested density (veh/km)	1/km
`v_1`	out	uncongested speed (km/h)	km/h

symbol		meaning	SI unit
`k_2`	out	congested density (veh/km)	1/km
`v_2`	out	congested speed (km/h)	km/h

Applies when: 0 < q <= vf kj / 4.

```
$ transportation solve transport_greenshields_speed_at_flow "vf=27.77777778 m/s" "kj=120 1/km"
"q=2000 1/h"
k_1 = 25.35898385 1/km
v_1 = 78.86751347 km/h
k_2 = 94.64101615 1/km
v_2 = 21.13248654 km/h
  by: Greenshields model: densities and speeds at a given flow  (k = (kj/2)(1 -/+ sqrt(1 - 4q/(vf
kj))), v = vf (1 - k/kj))
  vf = 27.77777778 m/s
  kj = 0.12 1/m
  q = 0.5555555556 1/s
  assumes: 0 < q <= vf kj / 4
```

What the worked example means: This calculation determines the two possible traffic states that support the given flow rate within the Greenshields model. Starting with a free-flow speed of 27.7778 m/s and jam density of 120 1/km, the densities and speeds corresponding to a flow of 2000 1/h are solved for. The result identifies an uncongested state with low density 25.359 1/km and high speed 78.8675 km/h, alongside a congested state with high density 94.641 1/km and low speed 21.1325 km/h. This dual solution illustrates that the same volume of traffic can occur under vastly different conditions, depending on whether drivers are moving freely or are in heavy, slow-moving traffic.

Probability of a headway longer than h (negative exponential), `transport_headway_probability`

Probability that the time headway between random vehicles at flow q is at least h (and shorter than h), for the negative exponential distribution or the shifted one with a minimum headway tau.

$$P(H \geq h) = e^{-(h-\tau)/(\bar{h}-\tau)},\ \bar{h} = 1/q$$

symbol		meaning	SI unit
`q`	in	traffic flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`h`	in	headway	s
`tau`	in	minimum headway (default 0) (optional)	s
`P_ge`	out	P(headway >= h)	,
`P_lt`	out	P(headway < h)	,

Applies when: q > 0; tau < 1/q.

```
$ transportation solve transport_headway_probability "q=600 1/h" "h=8 s"
P_ge = 0.2635971381
P_lt = 0.7364028619
  by: Probability of a headway longer than h (negative exponential)  (P(H >= h) = exp(-(h -
tau)/(1/q - tau)))
  q = 0.1666666667 1/s
  h = 8 s
  assumes: q > 0; tau < 1/q
```

In the worked example, q = 600 1/h, h = 8 s go through the equation above and give P_ge = 0.2635971381, P_lt = 0.7364028619. It applies only when q > 0; tau < 1/q: outside that it says what is wrong instead of returning a number.

Probability of n vehicle arrivals in an interval (Poisson), `transport_poisson_arrivals`

Probability that exactly n, at most n and at least n vehicles arrive in an interval t when arrivals are random (Poisson) at the flow q.

$$P(n) = \frac{(qt)^n e^{-qt}}{n!}$$

symbol		meaning	SI unit
`q`	in	traffic flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`t`	in	interval length	s
`n`	in	number of arrivals (whole number)	,
`P_n`	out	P(exactly n arrivals)	,
`P_le`	out	P(at most n arrivals)	,
`P_ge`	out	P(at least n arrivals)	,

Applies when: q > 0; t > 0; n a whole number >= 0.

```
$ transportation solve transport_poisson_arrivals "q=360 1/h" "t=30 s" n=2
P_n = 0.2240418077
P_le = 0.4231900811
P_ge = 0.8008517265
  by: Probability of n vehicle arrivals in an interval (Poisson)  (P(n) = (q t)^n e^(-q t) / n!)
  q = 0.1 1/s
  t = 30 s
  assumes: q > 0; t > 0; n a whole number >= 0
```

What the worked example means: This calculation determines the likelihood of observing specific numbers of vehicle arrivals within a defined time window, assuming random and independent traffic behavior. Using a flow rate of 360 1/h and an interval length of 30 s, the model computes that the probability of exactly 2 arrivals is 0.224042. It also provides the cumulative probabilities for at most 2 arrivals, which is 0.42319, and for at least 2 arrivals, which is 0.800852. These values help practitioners assess the variability of traffic demand over short periods.

Queue and delay at a temporary bottleneck (incident, lane closure), `transport_bottleneck_queue`

Deterministic queue when the capacity of a road falls from mu to mu_r for a time t_r (an incident or a work zone) while vehicles arrive at lam: the longest queue, how long the queue lasts and the total delay.

$$Q_{max} = (\lambda - \mu_r)t_r, \quad T = t_r + \frac{Q_{max}}{\mu - \lambda}, \quad D = \tfrac{1}{2} Q_{max} T$$

symbol		meaning	SI unit
`lam`	in	arrival flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`mu`	in	normal capacity (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`mu_r`	in	reduced capacity (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`t_r`	in	duration of the reduction	s
`Q_max`	out	longest queue (veh)	,
`T`	out	duration of the queue from the start of the reduction (s)	s
`T_min`	out	duration of the queue (min)	min
`D_total`	out	total delay (veh h)	h

Applies when: mu_r < lam < mu.

```
$ transportation solve transport_bottleneck_queue "lam=3200 1/h" "mu=4000 1/h" "mu_r=1800 1/h"
"t_r=1800 s"
Q_max = 700
T = 4950 s
T_min = 82.5 min
D_total = 481.25 h
  by: Queue and delay at a temporary bottleneck (incident, lane closure)  (Q = (lam - mu_r) t_r, T
= t_r + Q/(mu - lam), total delay = Q T / 2)
  lam = 0.8888888889 1/s
  mu = 1.111111111 1/s
  mu_r = 0.5 1/s
  t_r = 1800 s
  assumes: mu_r < lam < mu
```

What the worked example means: This calculation determines the impact of a temporary capacity reduction on traffic flow. Given an arrival rate of 3200 1/h, normal capacity of 4000 1/h, and reduced capacity of 1800 1/h over a duration of 1800 s, the model computes the peak queue length as 700 vehicles. The queue persists for 82.5 min minutes from the start of the incident, resulting in a total delay of 481.25 h vehicle hours. These values help quantify the congestion caused by incidents or work zones under steady-state conditions.

Queue from arrival counts and capacity per interval, `transport_interval_queue`

Vehicles queued at the end of each counting interval when the arrivals per interval are served up to the capacity per interval (deterministic queue, first in first out), the longest queue and the vehicles served per interval.

$$Q_k = \max\left(0,\ Q_{k-1} + a_k - c_k\right)$$

symbol		meaning	SI unit
`arrivals`	in	arrivals in each interval (veh)	,
`capacity`	in	capacity of each interval (veh), or one value for all	,
`queue`	out	queue at the end of each interval (veh)	,
`Q_max`	out	longest queue (veh)	,
`served`	out	vehicles served in each interval (veh)	,

Applies when: counts >= 0; capacity: one value or one per interval.

```
$ transportation solve transport_interval_queue "arrivals=[90, 120, 140, 110, 80, 60]" "capacity=
[100]"
queue = [0.0,20.0,60.0,70.0,50.0,10.0]
Q_max = 70
served = [90.0,100.0,100.0,100.0,100.0,100.0]
  by: Queue from arrival counts and capacity per interval  (Q_k = max(0, Q_{k-1} + a_k - c_k))
  assumes: counts >= 0; capacity: one value or one per interval
```

What the worked example means: This calculation determines the number of vehicles waiting in line at the end of each time interval by tracking how many arrive versus how many can depart. The result, shown as [0, 20, 60, 70, 50, 10], reflects the accumulation of unserved vehicles when arrivals exceed capacity, while [90, 100, 100, 100, 100, 100] indicates the throughput limited by the available capacity. The peak congestion level is captured by 70, which highlights the worst-case wait during the observed period.

Shockwave speed between two traffic states, `transport_shockwave_speed`

Speed of the boundary (shockwave) between an upstream traffic state (q1, k1) and a downstream state (q2, k2): u = (q2 - q1) / (k2 - k1), negative when it moves upstream (for example the back of a queue).

$$u_w = \frac{q_2 - q_1}{k_2 - k_1}$$

symbol		meaning	SI unit
`q1`	in	upstream flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`k1`	in	upstream density (veh/km in questions; veh/m in solve(), or '25 1/km')	1/m
`q2`	in	downstream flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`k2`	in	downstream density (veh/km in questions; veh/m in solve(), or '25 1/km')	1/m
`u`	out	shockwave speed (km/h), positive downstream	km/h
`u_ms`	out	shockwave speed (m/s)	m/s
`direction`	out	upstream, downstream or stationary	,

Applies when: k1, k2 > 0, q1, q2 >= 0; the denser state is the slower one.

```
$ transportation solve transport_shockwave_speed "q1=1800 1/h" "k1=25 1/km" "q2=1200 1/h" "k2=80
1/km"
u = -10.90909091 km/h
u_ms = -3.03030303 m/s
direction = upstream
  by: Shockwave speed between two traffic states  (u = (q2 - q1) / (k2 - k1))
  q1 = 0.5 1/s
  k1 = 0.025 1/m
  q2 = 0.3333333333 1/s
  k2 = 0.08 1/m
  assumes: k1, k2 > 0, q1, q2 >= 0; the denser state is the slower one
```

What the worked example means: This calculation determines the velocity of the traffic shockwave by comparing the upstream flow and density with the downstream values. Using the inputs 1800 1/h for upstream flow, 25 1/km for upstream density, 1200 1/h for downstream flow, and 80 1/km for downstream density, the resulting wave speed is -10.9091 km/h. This value represents how fast the transition between these two traffic states propagates along the road.

Shockwave speed in a Greenshields traffic stream, `transport_greenshields_shockwave`

Speed of the shockwave between two states of a Greenshields stream with densities k1 (upstream) < k2 (downstream): u = vf (1 - (k1 + k2) / kj).

$$u_w = v_f \left(1 - \frac{k_1 + k_2}{k_j}\right)$$

symbol		meaning	SI unit
`vf`	in	free-flow speed	m/s
`kj`	in	jam density (veh/km in questions; veh/m in solve(), or '25 1/km')	1/m
`k1`	in	upstream density (veh/km in questions; veh/m in solve(), or '25 1/km')	1/m
`k2`	in	downstream density (veh/km in questions; veh/m in solve(), or '25 1/km')	1/m
`u`	out	shockwave speed (km/h), positive downstream	km/h
`u_ms`	out	shockwave speed (m/s)	m/s

Applies when: 0 < k1 < k2 < kj.

```
$ transportation solve transport_greenshields_shockwave "vf=25 m/s" "kj=130 1/km" "k1=30 1/km"
"k2=90 1/km"
u = 6.923076923 km/h
u_ms = 1.923076923 m/s
  by: Shockwave speed in a Greenshields traffic stream  (u = vf (1 - (k1 + k2)/kj))
  vf = 25 m/s
  kj = 0.13 1/m
  k1 = 0.03 1/m
  k2 = 0.09 1/m
  assumes: 0 < k1 < k2 < kj
```

What the worked example means: This calculation determines the propagation speed of a discontinuity between two traffic states using the Greenshields model. Combining the free-flow speed 25 m/s with the

upstream density 30 1/km and downstream density 90 1/km, relative to jam density 130 1/km, shows that the boundary moves at 6.92308 km/h. This value indicates whether congestion is spreading backward or clearing forward along the road segment.

Travel-time reliability indices (buffer, planning time, travel time index), `transport_travel_time_reliability`

The buffer index (T95 - mean)/mean, the planning time index T95/T_free and the travel time index mean/T_free of observed travel times on a route (T95 the 95th percentile by linear interpolation). Times in minutes.

$$BI = \frac{T_{95} - \bar{T}}{\bar{T}}, \quad PTI = \frac{T_{95}}{T_{free}}, \quad TTI = \frac{\bar{T}}{T_{free}}$$

symbol		meaning	SI unit
`times`	in	observed travel times in min	,
`t_free`	in	free-flow travel time in min	,
`mean`	out	mean travel time (min)	,
`t95`	out	95th-percentile travel time (min)	,
`buffer_index`	out	buffer index (T95 - mean)/mean	,
`planning_time_index`	out	planning time index T95/T_free	,
`travel_time_index`	out	travel time index mean/T_free	,

Applies when: at least five positive travel times; T95 by linear interpolation (type 7).

```
$ transportation solve transport_travel_time_reliability "times=[22, 25, 24, 30, 27, 23, 35, 26,
24, 28]" t_free=18
mean = 26.4
t95 = 32.75
buffer_index = 0.240530303
planning_time_index = 1.819444444
travel_time_index = 1.466666667
  by: Travel-time reliability indices (buffer, planning time, travel time index)  (BI = (T95 -
mean)/mean; PTI = T95/T_free; TTI = mean/T_free)
  t_free = 18
  assumes: at least five positive travel times; T95 by linear interpolation (type 7)
```

What the worked example means: This calculation determines three key reliability metrics from observed travel times and a free-flow baseline. The mean travel time is 26.4 minutes, while the ninety-fifth percentile travel time is 32.75 minutes. From these, the buffer index is computed as 0.24053, representing the extra time needed to avoid lateness on most trips. The planning time index is 1.81944, showing the ratio of reliable travel time to free-flow conditions. Finally, the travel time index is 1.46667, indicating the average congestion level relative to free flow.

Underwood model: capacity and speed at capacity, `transport_underwood_capacity`

Capacity of a traffic stream following Underwood's model v = vf exp(-k / k0), with the speed and density at capacity.

$$q_{max} = \frac{v_f k_0}{e}, \quad v_m = \frac{v_f}{e}$$

symbol		meaning	SI unit
`vf`	in	free-flow speed	m/s
`k0`	in	optimum density k0 (veh/km in questions; veh/m in solve(), or '25 1/km')	1/m
`q_max`	out	capacity (veh/h)	1/h
`v_m`	out	speed at capacity (km/h)	km/h
`k_m`	out	density at capacity (veh/km)	1/km

Applies when: vf > 0; k0 > 0.

```
$ transportation solve transport_underwood_capacity "vf=27.77777778 m/s" "k0=50 1/km"
q_max = 1839.397206 1/h
v_m = 36.78794412 km/h
k_m = 50 1/km
  by: Underwood model: capacity and speed at capacity  (q = vf k exp(-k/k0) is largest at k = k0:
q_max = vf k0 / e, v = vf / e)
  vf = 27.77777778 m/s
  k0 = 0.05 1/m
  assumes: vf > 0; k0 > 0
```

What the worked example means: This calculation determines the maximum flow rate, known as capacity, for traffic following the Underwood exponential model. It computes this peak value using the free-flow speed and optimum density inputs, resulting in a capacity of 1839.4 1/h. The analysis also identifies the specific conditions at this peak, showing that the speed drops to 36.7879 km/h while the density reaches 50 1/km. These metrics provide essential performance indicators for understanding how traffic streams behave under idealized steady-state assumptions.

Traffic signals (5)

Green splits by equal degree of saturation (flow-ratio method), `transport_signal_green_splits`

Effective green time of each critical phase for a given cycle C and total lost time L, shared in proportion to the critical flow ratios y_i = v_i / s_i (equal degree of saturation), and the critical degree of saturation X_c. Greens in s.

$$g_i = \frac{y_i}{Y}(C - L), \quad X_c = \frac{Y C}{C - L}$$

symbol		meaning	SI unit
`C`	in	cycle length	s
`L`	in	total lost time per cycle	s
`y`	in	critical flow ratio v/s of each phase	,
`g`	out	effective green of each phase (s)	,
`X_c`	out	critical degree of saturation	,

Applies when: C > L; sum of y < 1.

```
$ transportation solve transport_signal_green_splits "C=90 s" "L=10 s" "y=[0.3, 0.25, 0.15]"
g = [34.2857142857,28.5714285714,17.1428571429]
X_c = 0.7875
  by: Green splits by equal degree of saturation (flow-ratio method)  (g_i = (y_i / Y)(C - L), X_c
= Y C / (C - L), Y = sum y_i)
  C = 90 s
  L = 10 s
  assumes: C > L; sum of y < 1
```

What the worked example means: This calculation determines the optimal effective green time for each signal phase by distributing the available green period proportionally to their critical flow ratios. Given a cycle length of 90 s seconds and total lost time of 10 s seconds, the method ensures that all phases operate at an equal degree of saturation. The computed effective green time is [34.2857, 28.5714, 17.1429] seconds per phase, which yields a uniform critical degree of saturation of 0.7875 across the intersection.

Optimum cycle length by Webster's formula, `transport_webster_optimum_cycle`

Webster's optimum cycle length C0 = (1.5 L + 5)/(1 - Y) of a fixed-time signal from the total lost time L (s) and the critical flow ratios y_i = v_i/s_i (Y = sum y_i), with the minimum cycle L/(1 - Y) and the total effective green. Times in s.

$$C_0 = \frac{1.5L + 5}{1 - Y}$$

symbol		meaning	SI unit
`L`	in	total lost time per cycle	s
`y`	in	critical flow ratio v/s of each phase	,
`C0`	out	Webster optimum cycle length (s)	s
`Y`	out	sum of the critical flow ratios	,
`C_min`	out	minimum cycle length L/(1 - Y) (s)	s
`green_total`	out	total effective green C0 - L (s)	s

Applies when: 0 < y_i < 1; Y < 0.95; fixed-time signal, Webster's delay model.

```
$ transportation solve transport_webster_optimum_cycle "L=12 s" "y=[0.3, 0.25, 0.15]"
C0 = 76.66666667 s
Y = 0.7
C_min = 40 s
green_total = 64.66666667 s
  by: Optimum cycle length by Webster's formula  (C0 = (1.5 L + 5)/(1 - Y), Y = sum of critical
y_i)
  L = 12 s
  assumes: 0 < y_i < 1; Y < 0.95; fixed-time signal, Webster's delay model
```

What the worked example means: This calculation determines the most efficient signal cycle duration for minimizing average delay at a fixed-time intersection. Using Webster's formula, the result depends on the total lost time per cycle, which is 12 s seconds, and the aggregate demand pressure represented by the sum

of critical flow ratios, 0.7. The resulting optimum cycle length is 76.6667 s seconds. This value balances the inefficiency caused by frequent yellow and red intervals against the benefits of longer green phases for traffic flow. Additionally, the minimum feasible cycle length required to serve this demand is 40 s seconds, leaving 64.6667 s seconds of effective green time available for movement.

Queue and uniform delay at a signal (deterministic D/D/1), `transport_signal_queue_uniform_delay`

Uniform arrivals at a fixed-time signal with saturation flow s, effective red r and effective green g (undersaturated): the largest queue, the time after the start of green until it clears, the total delay per cycle, the average (uniform) delay per vehicle and the share of vehicles that stop.

$$d = \frac{r^2}{2C\,(1-\lambda/s)} = \frac{C(1-g/C)^2}{2(1-\lambda/s)}$$

symbol		meaning	SI unit
`lam`	in	arrival flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`s`	in	saturation flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`r`	in	effective red	s
`g`	in	effective green	s
`Q_max`	out	largest queue (veh)	,
`t_clear`	out	time after the start of green to clear the queue (s)	s
`D_total`	out	total delay per cycle (veh s)	s
`d_uniform`	out	average delay per vehicle (s)	s
`P_stopped`	out	share of vehicles that stop	,

Applies when: lam < s; the queue clears within the green (lam C <= s g).

```
$ transportation solve transport_signal_queue_uniform_delay "lam=600 1/h" "s=1800 1/h" "r=40 s"
"g=30 s"
Q_max = 6.666666667
t_clear = 20 s
D_total = 200 s
d_uniform = 17.14285714 s
P_stopped = 0.8571428571
  by: Queue and uniform delay at a signal (deterministic D/D/1)  (Q = lam r, t_c = lam r/(s - lam),
D = lam r^2 / (2(1 - lam/s)), d = r^2 / (2C (1 - lam/s)))
  lam = 0.1666666667 1/s
  s = 0.5 1/s
  r = 40 s
  g = 30 s
  assumes: lam < s; the queue clears within the green (lam C <= s g)
```

What the worked example means: This calculation determines the uniform delay at a signalized intersection assuming steady arrivals. Given an arrival flow of 600 1/h and saturation flow of 1800 1/h, with effective red time 40 s and green time 30 s, the model computes that the largest queue reaches 6.66667 vehicles. The queue clears in 20 s seconds after green begins. The total delay accumulated over one cycle is 200 s vehicle-seconds, which translates to an average delay per vehicle of 17.1429 s seconds. Additionally, 0.857143 represents the proportion of vehicles that must stop during this cycle.

Queue at a signal by shockwave analysis, `transport_signal_queue_shockwave`

Kinematic-wave analysis of a signal: the queue grows upstream during the red at the shock speed between the arrival state (q_a, k_a) and jam density k_j, and from the start of green the discharge wave (saturation state q_s, k_s) chases it. The longest queue, the time it is reached (from the start of red) and both wave speeds.

$$t^* = \frac{w_2\, r}{w_2 - w_1}, \quad L_{max} = -w_1 t^*$$

symbol		meaning	SI unit
`q_a`	in	arrival flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`k_a`	in	arrival density (veh/km in questions; veh/m in solve(), or '25 1/km')	1/m
`k_j`	in	jam density (veh/km in questions; veh/m in solve(), or '25 1/km')	1/m
`q_s`	in	saturation (discharge) flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`k_s`	in	density of the discharging stream (veh/km in questions; veh/m in solve(), or '25 1/km')	1/m
`r`	in	red time	s
`L_max`	out	longest queue (m)	m
`t_max`	out	time from the start of red to the longest queue (s)	s
`w_queue`	out	speed of the back of the queue (km/h)	km/h
`w_discharge`	out	speed of the discharge wave (km/h)	km/h

Applies when: k_a, k_s < k_j; the discharge wave is faster than the queue growth.

```
$ transportation solve transport_signal_queue_shockwave "q_a=1000 1/h" "k_a=20 1/km" "k_j=140 1/km"
"q_s=1800 1/h" "k_s=50 1/km" "r=40 s"
L_max = 158.7301587 m
t_max = 68.57142857 s
w_queue = -8.333333333 km/h
w_discharge = -20 km/h
  by: Queue at a signal by shockwave analysis  (w1 = -q_a/(k_j - k_a), w2 = -q_s/(k_j - k_s); they
meet at t = w2 r/(w2 - w1); L_max = -w1 t)
  q_a = 0.2777777778 1/s
  k_a = 0.02 1/m
  k_j = 0.14 1/m
  q_s = 0.5 1/s
  k_s = 0.05 1/m
  r = 40 s
  assumes: k_a, k_s < k_j; the discharge wave is faster than the queue growth
```

What the worked example means: Kinematic wave analysis determines that the longest queue extends 158.73 m upstream from the stop line. This peak occurs 68.5714 s seconds after the signal turns red, when the discharge wave catches up with the growing queue. The back of the queue travels at -8.33333 km/h and the discharge wave moves at -20 km/h.

Yellow change interval (kinematic, ITE), `transport_yellow_change_interval`

Minimum yellow change interval Y = t_r + v / (2a + 2gG) so that a driver who is just too close to stop can clear at the approach speed, and the critical stopping distance from the stop line.

$$Y = t_r + \frac{v}{2a + 2gG}$$

symbol		meaning	SI unit
`v`	in	approach speed	m/s
`t_r`	in	perception-reaction time	s
`a`	in	comfortable deceleration rate	m/s^2
`G`	in	grade, decimal (positive uphill) (optional)	,
`Y`	out	yellow change interval (s)	s
`x_c`	out	critical stopping distance (m)	m

Applies when: v > 0; a + g G > 0.

```
$ transportation solve transport_yellow_change_interval "v=13.88888889 m/s" "t_r=1 s" "a=3.05
m/s^2"
Y = 3.276867031 s
x_c = 45.5120421 m
  by: Yellow change interval (kinematic, ITE)  (Y = t_r + v / (2a + 2 g G); x_c = v t_r + v^2 / (2
(a + g G)))
  v = 13.88888889 m/s
  t_r = 1 s
  a = 3.05 m/s^2
  assumes: v > 0; a + g G > 0
```

What the worked example means: The minimum yellow change interval is 3.27687 s and the critical stopping distance is 45.512 m.

Traffic studies (2)

85th, 50th and 15th percentile spot speeds, `transport_spot_speed_percentiles`

Percentile speeds of a spot-speed sample (85th percentile for speed limits and design, median, 15th) by linear interpolation between ranks (position (n - 1) p / 100, as Excel PERCENTILE.INC). Speeds in km/h.

$$v_p = v_{(i)} + f\,(v_{(i+1)} - v_{(i)}), \quad i + f = (n-1)\,p/100$$

symbol		meaning	SI unit
`speeds`	in	spot speeds in km/h	,
`p85`	out	85th percentile speed (km/h)	km/h
`p50`	out	median speed (km/h)	km/h
`p15`	out	15th percentile speed (km/h)	km/h

Applies when: at least two positive speeds.

```
$ transportation solve transport_spot_speed_percentiles "speeds=[52, 61, 48, 70, 55, 63, 58, 66,
49, 72]"
p85 = 68.6 km/h
p50 = 59.5 km/h
p15 = 50.05 km/h
  by: 85th, 50th and 15th percentile spot speeds  (sort; the p-th percentile at position (n - 1) p
/ 100 by linear interpolation)
  assumes: at least two positive speeds
```

What the worked example means: This calculation determines the 85th, 50th, and 15th percentile spot speeds from the input sample of observed vehicle velocities. By sorting the data and applying linear interpolation at specific rank positions, the method identifies representative speed values for design and regulatory purposes. The resulting 85th percentile speed is 68.6 km/h, the median speed is 59.5 km/h, and the 15th percentile speed is 50.05 km/h. These figures summarize the distribution of driver behavior, with the higher percentiles often guiding speed limit setting and the lower percentiles indicating slower traffic conditions.

Time-mean and space-mean speed of spot speeds, `transport_time_and_space_mean_speed`

From spot speeds measured at a point: the time-mean speed (arithmetic mean), the space-mean speed (harmonic mean, the speed that matches travel times over a fixed length) and the sample standard deviation. Speeds in km/h.

$$\bar{v}_t = \frac{1}{n}\sum v_i, \quad \bar{v}_s = \frac{n}{\sum 1/v_i}$$

symbol		meaning	SI unit
`speeds`	in	spot speeds in km/h	,
`TMS`	out	time-mean speed (km/h)	km/h
`SMS`	out	space-mean speed (km/h)	km/h
`s`	out	sample standard deviation (km/h)	km/h

Applies when: at least two positive speeds.

```
$ transportation solve transport_time_and_space_mean_speed "speeds=[52, 61, 48, 70, 55, 63]"
TMS = 58.16666667 km/h
SMS = 57.25294992 km/h
s = 8.035338616 km/h
  by: Time-mean and space-mean speed of spot speeds  (TMS = sum v / n; SMS = n / sum(1/v); s =
sqrt(sum (v - TMS)^2 / (n - 1)))
  assumes: at least two positive speeds
```

What the worked example means: This calculation derives the time-mean speed, which is the arithmetic average of the spot speeds, resulting in 58.1667 km/h. It also computes the space-mean speed, defined as the harmonic mean that corresponds to the average travel time over a fixed distance, yielding 57.2529 km/h. Additionally, the sample standard deviation of the speed distribution is calculated as 8.03534 km/h, indicating how much individual vehicle speeds vary from the time-mean. These metrics are essential for understanding traffic flow characteristics from point measurements.

Travel demand (4)

All-or-nothing traffic assignment, `transport_all_or_nothing_assignment`

Loads every origin-destination demand onto its fastest path through a network of one-way links with travel times (min): the flow on each link (in the order given) and the total vehicle-hours travelled.

$$x_a = \sum_{rs} q_{rs}\,\delta_{a,p^*_{rs}}$$

symbol		meaning	SI unit
`network`	in	one-way links as 'A-B time' with travel times in min	,
`od`	in	trip demands as 'origin-destination trips' (veh)	,
`flows`	out	flow on each link in the order given (veh)	,
`VHT`	out	vehicle-hours travelled (veh h)	h

Applies when: positive link times; every trip pair connected.

```
$ transportation solve transport_all_or_nothing_assignment "network=A-B 5, B-A 5, A-C 3, C-A 3, B-D
4, D-B 4, C-D 7, D-C 7, B-C 1, C-B 1" "od=A-D 800, C-B 200, D-A 300"
flows = [0.0,0.0,800.0,300.0,800.0,300.0,0.0,0.0,300.0,1000.0]
VHT = 150 h
  by: All-or-nothing traffic assignment  (for each trip pair the fastest path (Dijkstra); its
demand is added to every link on it)
  assumes: positive link times; every trip pair connected
```

In the worked example, network = A-B (5), B-A (5), A-C (3), C-A (3), B-D (4), D-B (4), C-D (7), D-C (7), B-C (1), C-B (1), od = A-D (800), C-B (200), D-A (300) go through the equation above and give flows = [0, 0, 800, 300, 800, 300, 0, 0, 300, 1000], VHT = 150 h. It applies only when positive link times; every trip pair connected: outside that it says what is wrong instead of returning a number.

Doubly constrained gravity model of trip distribution, `transport_gravity_model`

Trips between zones T_ij = A_i O_i B_j D_j f(c_ij) that match both the productions O_i and the attractions D_j, with deterrence f(c) = exp(-beta c) or c^-beta, balanced by Furness iterations until every total matches to 1e-13.

$$T_{ij} = A_i O_i B_j D_j f(c_{ij}), \quad \sum_j T_{ij} = O_i,\ \sum_i T_{ij} = D_j$$

symbol		meaning	SI unit
`O`	in	trip productions of the origin zones	,
`D`	in	trip attractions of the destination zones	,
`costs`	in	travel cost (time) matrix, rows origins, in min	,
`beta`	in	deterrence parameter	,
`deterrence`	in	exponential (default) or power (optional)	,
`trips`	out	trip matrix (rows origins, columns destinations)	,

Applies when: sum of productions = sum of attractions; beta >= 0; power deterrence: positive costs.

```
$ transportation solve transport_gravity_model "O=[1000, 1500, 800]" "D=[1200, 900, 1200]" "costs=
[[5, 12, 20], [10, 6, 15], [18, 14, 7]]" beta=0.1
trips = [[585.291609215,216.598390059,198.110000726],[494.749778142,550.037708887,455.212512971],
[119.958612642,133.363901055,546.677486303]]
  by: Doubly constrained gravity model of trip distribution  (Furness (iterative proportional
fitting) of O_i D_j f(c_ij) to the row and column totals)
  beta = 0.1
  assumes: sum of productions = sum of attractions; beta >= 0; power deterrence: positive costs
```

What the worked example means: The computed trip matrix [[585.292, 216.598, 198.11], [494.75, 550.038, 455.213], [119.959, 133.364, 546.677]] balances origin productions [1000, 1500, 800] and destination attractions [1200, 900, 1200] using the deterrence parameter 0.1.

Growth-factor (Furness) balancing of a trip matrix, `transport_furness_balancing`

Scales a base-year trip matrix to future row totals (productions) and column totals (attractions) by the Furness (biproportional) method, iterated until every total matches to 1e-13.

$$T_{ij} = t_{ij}\, a_i\, b_j$$

symbol		meaning	SI unit
`base`	in	base-year trip matrix (rows origins)	,
`O`	in	target row totals (productions)	,
`D`	in	target column totals (attractions)	,
`trips`	out	balanced trip matrix	,

Applies when: sum of row totals = sum of column totals; non-negative base matrix with a positive entry in every row and column.

```
$ transportation solve transport_furness_balancing "base=[[100, 200, 150], [300, 100, 250], [50,
150, 200]]" "O=[600, 800, 500]" "D=[550, 600, 750]"
trips = [[128.005362822,275.799789296,196.194847882],[361.892088736,129.955182633,308.15272863],
[60.1025484415,194.24502807,245.652423488]]
  by: Growth-factor (Furness) balancing of a trip matrix  (alternately scale rows to O_i and
columns to D_j until both match)
  assumes: sum of row totals = sum of column totals; non-negative base matrix with a positive entry
in every row and column
```

What the worked example means: The Furness balancing method scales the base-year trip matrix to align with future production and attraction targets. By alternately adjusting row and column sums, the algorithm produces a new matrix where total trips equal [[128.005, 275.8, 196.195], [361.892, 129.955, 308.153], [60.1025, 194.245, 245.652]]. This result confirms that the internal distribution of travel patterns has been preserved while meeting the global constraints of [600, 800, 500] for origins and [550, 600, 750] for destinations.

Trip generation by multiple linear regression, `transport_trip_generation_regression`

Least-squares trip-generation model trips = b0 + b1 x1 + ... fitted to zonal data (rows of X: zones; columns: households, employment, cars, ...), its coefficient of determination, and optionally the trips predicted for a new

zone.

$$\hat{\mathbf{b}} = \arg\min \|\mathbf{y} - [\mathbf{1}\ X]\mathbf{b}\|^2$$

symbol		meaning	SI unit
`X`	in	zonal predictors: one row per zone	,
`y`	in	observed trips of each zone	,
`x_new`	in	predictors of a zone to forecast (optional) (optional)	,
`coef`	out	coefficients b0 (intercept), b1, ...	,
`r2`	out	coefficient of determination	,
`y_new`	out	forecast trips of the new zone	,

Applies when: more zones than coefficients; predictors not collinear.

```
$ transportation solve transport_trip_generation_regression "X=[[120, 300], [250, 420], [400, 380],
[180, 600], [520, 700], [300, 250], [450, 520]]" "y=[1450, 2300, 3100, 2050, 4200, 2200, 3600]"
"x_new=[350, 500]"
coef = [258.227672282,5.71819751176,1.38738416969]
r2 = 0.994433954
y_new = 2953.288886
  by: Trip generation by multiple linear regression  (ordinary least squares with an intercept
(Householder QR))
  assumes: more zones than coefficients; predictors not collinear
```

What the worked example means: This calculation determines the best-fit linear relationship between zonal characteristics and trip generation using ordinary least squares. The resulting coefficients, [258.228, 5.7182, 1.38738], quantify how changes in predictors like households or employment affect trip counts. The coefficient of determination, 0.994434, indicates the proportion of variance in observed trips explained by the model. If a new zone's data is provided, the forecasted trips are calculated as 2953.29 based on these fitted parameters.

Unsignalized intersections (1)

Capacity of a minor stream by gap acceptance, `transport_gap_acceptance_capacity`

Capacity of a minor (stop- or yield-controlled) movement crossing a random major stream of flow q, with critical gap t_c and follow-up time t_f: Harders' formula (as in the HCM) or Siegloch's.

$$c = \frac{q\, e^{-q t_c}}{1 - e^{-q t_f}}$$

symbol		meaning	SI unit
`q`	in	major-stream flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')	1/s
`t_c`	in	critical gap	s
`t_f`	in	follow-up time	s
`model`	in	harders (HCM, default) or siegloch (optional)	,
`c`	out	capacity of the minor movement (veh/h)	1/h

Applies when: q, t_c, t_f > 0; Siegloch: t_c > t_f / 2.

```
$ transportation solve transport_gap_acceptance_capacity "q=800 1/h" "t_c=6.5 s" "t_f=3.3 s"
c = 363.1010029 1/h
  by: Capacity of a minor stream by gap acceptance  (Harders: c = q e^(-q t_c) / (1 - e^(-q t_f));
Siegloch: c = e^(-q (t_c - t_f/2)) / t_f)
  q = 0.2222222222 1/s
  t_c = 6.5 s
  t_f = 3.3 s
  assumes: q, t_c, t_f > 0; Siegloch: t_c > t_f / 2
```

What the worked example means: This calculation determines the maximum hourly discharge rate for a stop- or yield-controlled minor road merging into a random major traffic stream. The result, 363.101 1/h, indicates how many vehicles can safely enter per hour based on the available gaps in the main road. This capacity depends on the major stream flow 800 1/h, which represents the number of vehicles passing on the main road, as well as the driver's required safety margins, defined by the critical gap 6.5 s and the follow-up time 3.3 s. As the major flow increases, fewer large gaps are available, reducing the minor road's ability to merge.

Vehicle dynamics (9)

Lateral load transfer in a steady turn, `transport_lateral_load_transfer`

Outer and inner wheel loads of a rigid vehicle (no suspension roll) cornering at lateral acceleration a_y, and the load transferred to the outer wheels.

$$\Delta W = \frac{m a_y h}{t}$$

symbol		meaning	SI unit
`m`	in	vehicle mass	kg
`a_y`	in	lateral acceleration	m/s^2
`h`	in	height of the centre of gravity	m
`t_w`	in	track width	m
`W_out`	out	load on the outer wheels (N)	N
`W_in`	out	load on the inner wheels (N)	N
`dW`	out	load transferred to the outer wheels (N)	N

Applies when: all inputs positive; the inner wheels keep contact.

```
$ transportation solve transport_lateral_load_transfer "m=1500 kg" "a_y=0.6 m/s^2" "h=0.55 m"
"t_w=1.55 m"
W_out = 7676.854839 N
W_in = 7038.145161 N
dW = 319.3548387 N
  by: Lateral load transfer in a steady turn  (dW = m a_y h / t, W_out = W/2 + dW, W_in = W/2 - dW)
  m = 1500 kg
  a_y = 0.6 m/s^2
  h = 0.55 m
  t_w = 1.55 m
  assumes: all inputs positive; the inner wheels keep contact
```

What the worked example means: This calculation determines how vertical weight shifts between the left and right sides of a vehicle during a steady turn. Using the vehicle mass, lateral acceleration, center of gravity height, and track width, it computes the specific load on the outer wheels as 7676.85 N and the inner wheels as 7038.15 N. The difference, known as load transfer, is 319.355 N, which directly impacts tire grip and stability.

Longitudinal weight transfer under braking and ideal brake proportioning, `transport_braking_weight_transfer`

Dynamic front and rear axle loads of a vehicle decelerating at a, the load transferred to the front axle, and the ideal front share of the braking force (both axles at the same friction use).

$$W_f = \frac{W l_r + m a h}{L}, \quad \Delta W = \frac{m a h}{L}$$

symbol		meaning	SI unit
`m`	in	vehicle mass	kg
`a`	in	deceleration	m/s^2
`h`	in	height of the centre of gravity	m
`L`	in	wheelbase	m
`l_f`	in	distance from the front axle to the centre of gravity	m
`W_f`	out	front axle load (N)	N
`W_r`	out	rear axle load (N)	N
`dW`	out	load transferred to the front axle (N)	N
`front_share`	out	ideal front brake share	,

Applies when: a > 0 (deceleration); 0 < l_f < L; the rear axle keeps contact.

```
$ transportation solve transport_braking_weight_transfer "m=1500 kg" "a=7 m/s^2" "h=0.55 m" "L=2.7
m" "l_f=1.2 m"
W_f = 10313.88889 N
W_r = 4401.111111 N
dW = 2138.888889 N
front_share = 0.7009098803
  by: Longitudinal weight transfer under braking and ideal brake proportioning  (W_f = (W l_r + m a
h) / L, dW = m a h / L, ideal front share = W_f / W)
  m = 1500 kg
  a = 7 m/s^2
  h = 0.55 m
  L = 2.7 m
  l_f = 1.2 m
  assumes: a > 0 (deceleration); 0 < l_f < L; the rear axle keeps contact
```

What the worked example means: This calculation determines how braking shifts weight forward, resulting in a front axle load of 10313.9 N and a rear axle load of 4401.11 N. The load transferred to the front axle is 2138.89 N. Based on these dynamic loads, the ideal proportion of total braking force that should be applied to the front wheels to achieve maximum deceleration without lockup is 0.70091. This ensures both axles use their available friction equally.

Quarter-car model: body and wheel-hop natural frequencies, `transport_quarter_car_frequencies`

Undamped natural frequencies of the two-degree-of-freedom quarter-car ride model: the body (bounce) mode and the wheel-hop mode, from the sprung and unsprung masses, suspension and tyre stiffnesses.

$$m_s m_u \omega^4 - [k_s m_u + (k_s + k_t) m_s] \omega^2 + k_s k_t = 0$$

symbol		meaning	SI unit
`m_s`	in	sprung mass (a quarter of the body)	kg
`m_u`	in	unsprung mass (wheel)	kg
`k_s`	in	suspension spring stiffness	N/m
`k_t`	in	tyre vertical stiffness	N/m
`f_body`	out	body (bounce) natural frequency (Hz)	Hz
`f_wheel`	out	wheel-hop natural frequency (Hz)	Hz

Applies when: all inputs positive.

```
$ transportation solve transport_quarter_car_frequencies "m_s=400 kg" "m_u=45 kg" "k_s=25000 N/m"
"k_t=200000 N/m"
f_body = 1.18543874 Hz
f_wheel = 11.26185411 Hz
  by: Quarter-car model: body and wheel-hop natural frequencies  (roots of m_s m_u w^4 - (k_s m_u +
(k_s + k_t) m_s) w^2 + k_s k_t = 0)
  m_s = 400 kg
  m_u = 45 kg
  k_s = 25000 N/m
  k_t = 200000 N/m
  assumes: all inputs positive
```

What the worked example means: This calculation determines the undamped natural frequencies for a quarter-car suspension model. It uses the sprung mass, unsprung mass, suspension stiffness, and tyre stiffness to solve for two distinct modes of vibration. The lower result, 1.18544 Hz, represents the oscillation rate of the vehicle body, which primarily influences ride comfort. The higher result, 11.2619 Hz, represents the oscillation rate of the wheel assembly, which affects road holding and traction. These values indicate how quickly the system returns to equilibrium after a disturbance in an idealized, frictionless scenario.

Quarter-car model: road-to-body transmissibility at a frequency, `transport_quarter_car_transmissibility`

Amplitude ratio between body (sprung-mass) motion and road displacement, and between wheel motion and road displacement, of the damped quarter-car model for a sinusoidal road input of frequency f.

$$\left(K + i\omega C - \omega^2 M\right)\mathbf{Z} = \begin{bmatrix} 0 \\ k_t \end{bmatrix} Z_r$$

symbol		meaning	SI unit
`m_s`	in	sprung mass (a quarter of the body)	kg
`m_u`	in	unsprung mass (wheel)	kg
`k_s`	in	suspension spring stiffness	N/m
`c_s`	in	suspension damping coefficient	kg/s
`k_t`	in	tyre vertical stiffness	N/m
`f`	in	road input frequency	Hz
`T_body`	out	body displacement / road displacement	,
`T_wheel`	out	wheel displacement / road displacement	,

Applies when: masses and stiffnesses positive; c_s >= 0.

```
$ transportation solve transport_quarter_car_transmissibility "m_s=400 kg" "m_u=45 kg" "k_s=25000
N/m" "c_s=1800 kg/s" "k_t=200000 N/m" "f=1.2 Hz"
T_body = 2.396334319
T_wheel = 1.159037238
  by: Quarter-car model: road-to-body transmissibility at a frequency  (solve the 2x2 complex
dynamic-stiffness system [K + i w C - w^2 M] Z = [0, k_t] for the road input)
  m_s = 400 kg
  m_u = 45 kg
  k_s = 25000 N/m
  c_s = 1800 kg/s
  k_t = 200000 N/m
  f = 1.2 Hz
  assumes: masses and stiffnesses positive; c_s >= 0
```

What the worked example means: This calculation determines how much vehicle motion results from a specific sinusoidal road input. By solving the dynamic equations for the quarter-car model, it yields two amplitude ratios: one for the sprung mass (body) and one for the unsprung mass (wheel). The result shows that at a driving frequency of 1.2 Hz Hz, with a suspension damping coefficient of 1800 kg/s kg/s, the body displacement is 2.39633 times the road displacement, while the wheel displacement is 1.15904 times the road displacement.

Rollover threshold: static stability factor and rollover speed on a curve, `transport_rollover_threshold`

Static stability factor SSF = t / (2h) of a rigid vehicle and the speed at which it would tip over on a curve of radius R with superelevation e (enough friction assumed for the tyres not to slide).

$$SSF = \frac{t}{2h}, \quad \frac{v^2}{gR} = \frac{SSF + e}{1 - SSF\, e}$$

symbol		meaning	SI unit
`t_w`	in	track width	m
`h`	in	height of the centre of gravity	m
`R`	in	curve radius	m
`e`	in	superelevation rate, decimal (optional)	,
`SSF`	out	static stability factor t / (2h) (rollover threshold in g)	,
`v_roll`	out	rollover speed on the curve (m/s)	m/s
`v_roll_kmh`	out	rollover speed (km/h)	km/h

Applies when: SSF e < 1; tyre friction sufficient (sliding is not checked).

```
$ transportation solve transport_rollover_threshold "t_w=1.55 m" "h=0.6 m" "R=100 m"
SSF = 1.291666667
v_roll = 35.59669929 m/s
v_roll_kmh = 128.1481174 km/h
  by: Rollover threshold: static stability factor and rollover speed on a curve  (SSF = t / (2h);
v^2 / (g R) = (SSF + e) / (1 - SSF e))
  t_w = 1.55 m
  h = 0.6 m
  R = 100 m
  assumes: SSF e < 1; tyre friction sufficient (sliding is not checked)
```

What the worked example means: The static stability factor is 1.29167 and the rollover speed on this curve is 128.148 km/h.

Steady-state cornering: yaw rate, lateral acceleration, radius, sideslip, `transport_steady_state_cornering`

Steady turning response of the linear bicycle model at speed u to a front-wheel steer angle delta: yaw rate, lateral acceleration, turn radius and the sideslip angle at the centre of gravity (small angles).

$$\frac{r}{\delta} = \frac{u}{L + Ku^2}$$

symbol		meaning	SI unit
`m`	in	vehicle mass	kg
`L`	in	wheelbase	m
`l_f`	in	distance from the front axle to the centre of gravity	m
`C_f`	in	cornering stiffness of the front axle	N/rad
`C_r`	in	cornering stiffness of the rear axle	N/rad

symbol		meaning	SI unit
`u`	in	forward speed	m/s
`delta`	in	front-wheel steer angle (degrees in questions; rad in solve(), or '2 deg')	rad
`r`	out	yaw rate (rad/s)	rad/s
`a_y`	out	lateral acceleration (m/s^2)	m/s^2
`R`	out	turn radius (m)	m
`beta`	out	sideslip angle at the centre of gravity (degrees)	deg

Applies when: below the critical speed; delta != 0.

```
$ transportation solve transport_steady_state_cornering "m=1500 kg" "L=2.7 m" "l_f=1.2 m"
"C_f=90000 N/rad" "C_r=85000 N/rad" "u=20 m/s" "delta=2 deg"
r = 0.2137280402 rad/s
a_y = 4.274560804 m/s^2
R = 93.57686517 m
beta = -1.002467818 deg
  by: Steady-state cornering: yaw rate, lateral acceleration, radius, sideslip  (r = u delta / (L +
K u^2), a_y = u r, R = u / r, beta = delta (l_r - m l_f u^2 / (L C_r)) / (L + K u^2))
  m = 1500 kg
  L = 2.7 m
  l_f = 1.2 m
  C_f = 90000 N/rad
  C_r = 85000 N/rad
  u = 20 m/s
  delta = 0.03490658504 rad
  assumes: below the critical speed; delta != 0
```

What the worked example means: This calculation determines the vehicle's constant-radius turning behavior at speed 20 m/s with a steer angle of 2 deg. The resulting yaw rate is 0.213728 rad/s, which corresponds to a lateral acceleration of 4.27456 m/s^2 and a path radius of 93.5769 m. The sideslip angle at the center of gravity is computed as -1.00247 deg, indicating how much the vehicle body deviates from its velocity vector during this steady maneuver.

Steer angle needed for a turn of given radius (bicycle model), `transport_steer_angle_for_radius`

Front-wheel steer angle the linear bicycle model needs to hold a turn of radius R at speed u (Ackermann angle L/R plus the understeer term K u^2 / R).

$$\delta = \frac{L}{R} + K\frac{u^2}{R}$$

symbol		meaning	SI unit
`m`	in	vehicle mass	kg
`L`	in	wheelbase	m
`l_f`	in	distance from the front axle to the centre of gravity	m
`C_f`	in	cornering stiffness of the front axle	N/rad
`C_r`	in	cornering stiffness of the rear axle	N/rad
`u`	in	forward speed	m/s

symbol		meaning	SI unit
`R`	in	turn radius	m
`delta`	out	front-wheel steer angle (degrees)	deg

Applies when: below the critical speed.

```
$ transportation solve transport_steer_angle_for_radius "m=1500 kg" "L=2.7 m" "l_f=1.2 m"
"C_f=90000 N/rad" "C_r=85000 N/rad" "u=20 m/s" "R=150 m"
delta = 1.247691536 deg
  by: Steer angle needed for a turn of given radius (bicycle model)  (delta = L / R + K u^2 / R)
  m = 1500 kg
  L = 2.7 m
  l_f = 1.2 m
  C_f = 90000 N/rad
  C_r = 85000 N/rad
  u = 20 m/s
  R = 150 m
  assumes: below the critical speed
```

What the worked example means: This calculation determines the front-wheel steering angle needed to maintain a turn of radius 150 m at a forward speed of 20 m/s. The result, 1.24769 deg, combines the geometric Ackermann angle with a dynamic correction for understeer. A larger wheelbase or higher mass increases the required angle, while greater cornering stiffness at the front reduces it.

Understeer gradient and characteristic or critical speed (bicycle model), `transport_understeer_gradient`

Understeer gradient K of the linear single-track model from axle cornering stiffnesses and the centre-of-gravity position, in rad/(m/s^2) and deg/g, whether the vehicle understeers or oversteers, and its characteristic speed (understeer: the steer angle is twice the Ackermann angle) or critical speed (oversteer: it becomes unstable).

$$K = \frac{m}{L}\left(\frac{l_r}{C_f} - \frac{l_f}{C_r}\right), \quad v_{ch} = \sqrt{L/K}$$

symbol		meaning	SI unit
`m`	in	vehicle mass	kg
`L`	in	wheelbase	m
`l_f`	in	distance from the front axle to the centre of gravity	m
`C_f`	in	cornering stiffness of the front axle	N/rad
`C_r`	in	cornering stiffness of the rear axle	N/rad
`K`	out	understeer gradient (rad s^2/m)	rad*s^2/m
`K_deg_per_g`	out	understeer gradient (deg/g)	deg
`behaviour`	out	understeer or oversteer	,
`v_c`	out	characteristic or critical speed (m/s)	m/s
`v_c_kmh`	out	characteristic or critical speed (km/h)	km/h

Applies when: 0 < l_f < L; positive cornering stiffnesses; not exactly neutral steer.

```
$ transportation solve transport_understeer_gradient "m=1500 kg" "L=2.7 m" "l_f=1.2 m" "C_f=90000
N/rad" "C_r=85000 N/rad"
K = 0.001416122004 rad*s^2/m
K_deg_per_g = 0.7959619566 deg
behaviour = understeer
v_c = 43.66480716 m/s
v_c_kmh = 157.1933058 km/h
  by: Understeer gradient and characteristic or critical speed (bicycle model)  (K = m (l_r / (L
C_f) - l_f / (L C_r)); v_ch = sqrt(L / K) (K > 0), v_crit = sqrt(-L / K) (K < 0))
  m = 1500 kg
  L = 2.7 m
  l_f = 1.2 m
  C_f = 90000 N/rad
  C_r = 85000 N/rad
  assumes: 0 < l_f < L; positive cornering stiffnesses; not exactly neutral steer
```

What the worked example means: This calculation determines the handling characteristics of the vehicle by computing the understeer gradient from the mass, wheelbase, and axle cornering stiffnesses. The resulting gradient is 0.00141612 rad*s^2/m, which indicates that the vehicle behaves as a understeer car. Based on this stability characteristic, the relevant speed threshold is calculated as 43.6648 m/s. This value represents either the characteristic speed where steering effort doubles or the critical speed at which the vehicle becomes unstable.

Yaw natural frequency and damping of the bicycle model, `transport_yaw_natural_frequency`

Undamped natural frequency and damping ratio of the lateral-yaw motion of the linear bicycle model at speed u (from the characteristic equation of its two states), below the critical speed.

$$\omega_n^2 = \frac{C_f C_r L^2}{m I_z u^2}\left(1 + \frac{K u^2}{L}\right)$$

symbol		meaning	SI unit
`m`	in	vehicle mass	kg
`Iz`	in	yaw moment of inertia	kg*m^2
`L`	in	wheelbase	m
`l_f`	in	distance from the front axle to the centre of gravity	m
`C_f`	in	cornering stiffness of the front axle	N/rad
`C_r`	in	cornering stiffness of the rear axle	N/rad
`u`	in	forward speed	m/s
`wn`	out	undamped natural frequency (rad/s)	rad/s
`zeta`	out	damping ratio	,
`f_n`	out	undamped natural frequency (Hz)	Hz

Applies when: below the critical speed.

```
$ transportation solve transport_yaw_natural_frequency "m=1500 kg" "Iz=2500 kg*m^2" "L=2.7 m"
"l_f=1.2 m" "C_f=90000 N/rad" "C_r=85000 N/rad" "u=20 m/s"
wn = 6.706638502 rad/s
zeta = 0.913299064
f_n = 1.067394669 Hz
  by: Yaw natural frequency and damping of the bicycle model  (wn^2 = C_f C_r L^2 / (m Iz u^2) (1 +
K u^2 / L), 2 zeta wn = (C_f + C_r)/(m u) + (C_f l_f^2 + C_r l_r^2)/(Iz u))
  m = 1500 kg
  Iz = 2500 kg*m^2
  L = 2.7 m
  l_f = 1.2 m
  C_f = 90000 N/rad
  C_r = 85000 N/rad
  u = 20 m/s
  assumes: below the critical speed
```

What the worked example means: This calculation determines the dynamic response characteristics of a vehicle's yaw motion. Using inputs such as mass, inertia, wheelbase, and tire stiffness at speed 20 m/s, it computes an undamped natural frequency of 6.70664 rad/s and a damping ratio of 0.913299. These values indicate how quickly the vehicle oscillates and returns to straight-line travel after a disturbance.

Vehicle performance (8)

Acceleration time and distance under constant engine power, `transport_acceleration_under_power`

Time and distance for a vehicle of mass m to change speed from v1 to v2 with constant power P at the wheels (tractive force eta P / v) against rolling, aerodynamic and grade resistance. A truck slowing on an upgrade towards its crawl speed is covered too (v2 < v1); both speeds must lie on the same side of the speed at which the power balances the resistance.

$$t = \int_{v_1}^{v_2} \frac{\gamma m \, dv}{\eta P / v - F_0 - K v^2}$$

symbol		meaning	SI unit
`m`	in	vehicle mass	kg
`P`	in	engine power	W
`v1`	in	initial speed	m/s
`v2`	in	final speed	m/s
`C_D`	in	drag coefficient	,
`A`	in	frontal area	m^2
`rho`	in	air density (default 1.2 kg/m^3) (optional)	kg/m^3
`f_r`	in	rolling resistance coefficient (default 0.015) (optional)	,
`G`	in	grade, decimal (positive uphill) (optional)	,
`eta`	in	drivetrain efficiency (default 1) (optional)	,
`gamma`	in	mass factor for rotating parts (default 1) (optional)	,
`t`	out	time to change speed (s)	s
`d`	out	distance travelled (m)	m

Applies when: v1, v2 > 0 on the same side of the equilibrium speed; eta <= 1.

```
$ transportation solve transport_acceleration_under_power "m=1500 kg" "P=75000 W" "v1=5.555555556
m/s" "v2=27.77777778 m/s" "C_D=0.32 1" "A=2.2 m^2"
t = 8.320252897 s
d = 162.0537102 m
  by: Acceleration time and distance under constant engine power  (t = integral of gamma m dv /
(eta P / v - F0 - K v^2), d = integral of gamma m v dv / (...) (numerical quadrature))
  m = 1500 kg
  P = 75000 W
  v1 = 5.555555556 m/s
  v2 = 27.77777778 m/s
  C_D = 0.32
  A = 2.2 m^2
  assumes: v1, v2 > 0 on the same side of the equilibrium speed; eta <= 1
```

What the worked example means: The vehicle takes 8.32025 s to change speed from 5.55556 m/s to 27.7778 m/s, covering a distance of 162.054 m.

Coast-down time and distance (rolling and aerodynamic resistance), `transport_coast_down`

Time and distance for a vehicle coasting in neutral to slow from v1 to v2 under rolling resistance f_r, the grade and aerodynamic drag rho C_D A v^2 / 2.

$$t = \frac{\gamma m}{\sqrt{K F_0}} \left[\arctan\left(v_1 \sqrt{K/F_0} \right) - \arctan\left(v_2 \sqrt{K/F_0} \right) \right]$$

symbol		meaning	SI unit
`m`	in	vehicle mass	kg
`v1`	in	initial speed	m/s
`v2`	in	final speed	m/s
`C_D`	in	drag coefficient	,
`A`	in	frontal area	m^2
`rho`	in	air density (default 1.2 kg/m^3) (optional)	kg/m^3
`f_r`	in	rolling resistance coefficient (default 0.012) (optional)	,
`G`	in	grade, decimal (positive uphill) (optional)	,
`gamma`	in	mass factor (default 1) (optional)	,
`t`	out	coast-down time (s)	s
`d`	out	coast-down distance (m)	m

Applies when: v2 < v1; F0 = W (f_r cos(theta) + sin(theta)) > 0.

```
$ transportation solve transport_coast_down "m=1400 kg" "v1=27.77777778 m/s" "v2=16.66666667 m/s"
"C_D=0.32 1" "A=2.2 m^2"
t = 42.25012979 s
d = 917.2484984 m
  by: Coast-down time and distance (rolling and aerodynamic resistance)  (t = gamma m / sqrt(K F0)
(atan(v1 sqrt(K/F0)) - atan(v2 sqrt(K/F0))); d = gamma m / (2K) ln((F0 + K v1^2)/(F0 + K v2^2)))
  m = 1400 kg
  v1 = 27.77777778 m/s
  v2 = 16.66666667 m/s
  C_D = 0.32
  A = 2.2 m^2
  assumes: v2 < v1; F0 = W (f_r cos(theta) + sin(theta)) > 0
```

What the worked example means: This calculation determines the time and distance required for a vehicle to decelerate naturally when in neutral gear. By accounting for rolling resistance, aerodynamic drag, and gravity on a slope, it yields a coast-down time of 42.2501 s and a distance of 917.248 m as the speed drops from 27.7778 m/s to 16.6667 m/s. These values help engineers assess passive braking characteristics without relying on engine or friction brakes.

Distance and time to accelerate (acceleration lane length), `transport_acceleration_distance`

Distance and time for a vehicle to accelerate from v1 to v2 at a constant acceleration a (for example the length of an acceleration lane).

$$d = \frac{v_2^2 - v_1^2}{2a}$$

symbol		meaning	SI unit
`v1`	in	initial speed	m/s
`v2`	in	final speed	m/s
`a`	in	acceleration	m/s^2
`d`	out	distance travelled while accelerating (m)	m
`t`	out	time taken (s)	s

Applies when: v2 > v1 >= 0; a > 0.

```
$ transportation solve transport_acceleration_distance "v1=11.11111111 m/s" "v2=27.77777778 m/s"
"a=1.2 m/s^2"
d = 270.0617285 m
t = 13.88888889 s
  by: Distance and time to accelerate (acceleration lane length)  (d = (v2^2 - v1^2) / (2a), t =
(v2 - v1) / a)
  v1 = 11.11111111 m/s
  v2 = 27.77777778 m/s
  a = 1.2 m/s^2
  assumes: v2 > v1 >= 0; a > 0
```

What the worked example means: This calculation determines the distance and time needed for a vehicle to accelerate from an initial speed of 11.1111 m/s to a final speed of 27.7778 m/s under a constant acceleration of 1.2 m/s^2. The resulting distance of 270.062 m represents the physical space required, such as the length

of an acceleration lane, while the time of 13.8889 s indicates how long this maneuver takes. These values are critical for assessing whether a merge or entry is feasible given the available road infrastructure.

Maximum tractive force and acceleration limited by tyre friction,
`transport_traction_limited_acceleration`

The largest tractive force the road can transmit (rear-, front- or all-wheel drive), the resulting maximum acceleration and the dynamic axle loads, with weight transfer from a centre of gravity at height h and rolling resistance f_r (aerodynamic drag neglected, level road).

$$F_{max} = \frac{\mu W (l_f - f_{rl} h)/L}{1 - \mu h / L}$$

symbol		meaning	SI unit
`m`	in	vehicle mass	kg
`L`	in	wheelbase	m
`l_f`	in	distance from the front axle to the centre of gravity	m
`h`	in	height of the centre of gravity	m
`mu`	in	tyre-road friction coefficient	,
`f_r`	in	rolling resistance coefficient (default 0) (optional)	,
`drive`	in	driven axles: rear, front or all (optional)	,
`F_max`	out	maximum tractive force (N)	N
`a_max`	out	maximum acceleration (m/s^2)	m/s^2
`W_f`	out	front axle load (N)	N
`W_r`	out	rear axle load (N)	N

Applies when: 0 < l_f < L; mu h < L for rear-wheel drive; both axles keep contact.

```
$ transportation solve transport_traction_limited_acceleration "m=1500 kg" "L=2.7 m" "l_f=1.1 m"
"h=0.55 m" "mu=0.8 1" f_r=0.012 drive=rear
F_max = 5695.356106 N
a_max = 3.679184071 m/s^2
W_f = 7595.804867 N
W_r = 7119.195133 N
  by: Maximum tractive force and acceleration limited by tyre friction  (RWD: F = mu W (l_f - f_r
h)/L / (1 - mu h / L); FWD: F = mu W (l_r + f_r h)/L / (1 + mu h / L); AWD: F = mu W)
  m = 1500 kg
  L = 2.7 m
  l_f = 1.1 m
  h = 0.55 m
  mu = 0.8
  f_r = 0.012
  drive = rear
  assumes: 0 < l_f < L; mu h < L for rear-wheel drive; both axles keep contact
```

What the worked example means: This calculation determines the highest force the tyres can transmit to the road, given the vehicle mass 1500 kg, wheelbase 2.7 m, centre of gravity position 1.1 m and height 0.55 m, friction coefficient 0.8, rolling resistance 0.012, and drive configuration rear. The resulting maximum tractive

force is 5695.36 N, which leads to a maximum acceleration of 3.67918 m/s^2. During this acceleration, weight transfer shifts load from the front axle, now carrying 7595.8 N, to the rear axle, now carrying 7119.2 N.

Minimum safe following gap when the leader brakes, `transport_safe_following_gap`

Smallest initial gap that avoids a rear-end collision when a leader brakes at a_l and the follower, at the same initial speed, reacts after t_r and brakes at a_f (the closest approach can happen while both still move).

$$g_{min} = \max_{t} \left[x_f(t) - x_l(t) \right]$$

symbol		meaning	SI unit
`v`	in	initial speed of both vehicles	m/s
`t_r`	in	follower reaction time	s
`a_l`	in	deceleration of the leader	m/s^2
`a_f`	in	deceleration of the follower	m/s^2
`gap`	out	minimum safe initial gap (m)	m

Applies when: all inputs positive.

```
$ transportation solve transport_safe_following_gap "v=27.77777778 m/s" "t_r=1 s" "a_l=8 m/s^2"
"a_f=6 m/s^2"
gap = 43.85288066 m
  by: Minimum safe following gap when the leader brakes  (gap = max over time of (x_follower -
x_leader): at the end, or when the speeds are equal)
  v = 27.77777778 m/s
  t_r = 1 s
  a_l = 8 m/s^2
  a_f = 6 m/s^2
  assumes: all inputs positive
```

What the worked example means: This calculation determines the smallest distance needed between two vehicles traveling at the same speed to prevent a rear-end collision when the leading vehicle brakes. It accounts for the follower's reaction time and the differing deceleration rates of both vehicles. The result, 43.8529 m, represents the critical separation distance where the closest approach occurs either when speeds equalize or at the end of the braking event.

Overtaking manoeuvre: time and distance with constant acceleration, `transport_overtaking_manoeuvre`

A vehicle that starts at the speed v of the vehicle it passes accelerates at a until it has gained the relative distance s (initial gap + both lengths + final gap): the time, its distance travelled and its final speed.

$$t = \sqrt{\frac{2s}{a}}, \quad d = vt + \tfrac{1}{2}at^2$$

symbol		meaning	SI unit
`v`	in	speed of the overtaken vehicle (and initial speed of the passing vehicle)	m/s
`a`	in	acceleration of the passing vehicle	m/s^2

symbol		meaning	SI unit
`s`	in	relative distance to gain	m
`t`	out	time to complete the overtaking (s)	s
`d`	out	distance travelled by the passing vehicle (m)	m
`v_f`	out	final speed of the passing vehicle (m/s)	m/s

Applies when: v >= 0; a > 0; s > 0.

```
$ transportation solve transport_overtaking_manoeuvre "v=19.44444444 m/s" "a=1.5 m/s^2" "s=60 m"
t = 8.94427191 s
d = 233.9163982 m
v_f = 32.86085231 m/s
  by: Overtaking manoeuvre: time and distance with constant acceleration  (t = sqrt(2 s / a), d = v
t + a t^2 / 2)
  v = 19.44444444 m/s
  a = 1.5 m/s^2
  s = 60 m
  assumes: v >= 0; a > 0; s > 0
```

What the worked example means: This calculation determines the time and distance required for a vehicle to overtake another by accelerating at a constant rate. Starting with an initial speed equal to that of the vehicle being passed, the passing vehicle accelerates until it has gained the necessary relative distance. The result shows that the maneuver requires 8.94427 s seconds and covers 233.916 m meters, providing a basis for assessing the safety and feasibility of the pass under ideal conditions.

Top speed of a road vehicle, `transport_top_speed`

The steady top speed at which the power at the wheels eta P equals the power absorbed by rolling, aerodynamic and grade resistance: rho C_D A v^3/2 + m g (f_r cos(theta) + sin(theta)) v = eta P.

$$\tfrac{1}{2}\rho C_D A v^3 + mg\left(f_r\cos\theta + \sin\theta\right)v = \eta P$$

symbol		meaning	SI unit
`m`	in	vehicle mass	kg
`P`	in	engine power	W
`C_D`	in	drag coefficient	,
`A`	in	frontal area	m^2
`rho`	in	air density (default 1.2 kg/m^3) (optional)	kg/m^3
`f_r`	in	rolling resistance coefficient (default 0.015) (optional)	,
`G`	in	grade, decimal (positive uphill) (optional)	,
`eta`	in	drivetrain efficiency (default 1) (optional)	,
`v_max`	out	top speed (m/s)	m/s
`v_max_kmh`	out	top speed (km/h)	,

Applies when: m, P, C_D, A, rho > 0; 0 <= f_r <= 0.1; |G| <= 0.3; 0 < eta <= 1; power-limited (no gearing or traction limit).

```
$ transportation solve transport_top_speed "m=1500 kg" "P=100000 W" "C_D=0.3 1" "A=2.2 m^2"
v_max = 60.27021255 m/s
v_max_kmh = 216.9727652
  by: Top speed of a road vehicle  (solve rho C_D A v^3/2 + m g (f_r cos(theta) + sin(theta)) v =
eta P by bisection)
  m = 1500 kg
  P = 100000 W
  C_D = 0.3
  A = 2.2 m^2
  assumes: m, P, C_D, A, rho > 0; 0 <= f_r <= 0.1; |G| <= 0.3; 0 < eta <= 1; power-limited (no
gearing or traction limit)
```

What the worked example means: The calculated top speed is 60.2702 m/s (216.973).

Vehicle resistance forces and power at a speed, `transport_vehicle_resistance`

Rolling resistance f_r m g cos(theta), aerodynamic drag rho C_D A v^2/2 and grade resistance m g sin(theta) (tan theta = G) of a road vehicle at a speed, their sum and the power needed to overcome them.

$$F = f_r mg\cos\theta + \tfrac{1}{2}\rho C_D A v^2 + mg\sin\theta, \quad P = Fv$$

symbol		meaning	SI unit
`m`	in	vehicle mass	kg
`v`	in	speed	m/s
`C_D`	in	drag coefficient	,
`A`	in	frontal area	m^2
`rho`	in	air density (default 1.2 kg/m^3) (optional)	kg/m^3
`f_r`	in	rolling resistance coefficient (default 0.015) (optional)	,
`G`	in	grade, decimal (positive uphill) (optional)	,
`F_roll`	out	rolling resistance (N)	N
`F_aero`	out	aerodynamic drag (N)	N
`F_grade`	out	grade resistance (N)	N
`F_total`	out	total resistance (N)	N
`P`	out	power to overcome the resistance (W)	W
`P_kW`	out	power (kW)	,

Applies when: m, C_D, A, rho > 0; v >= 0; 0 <= f_r <= 0.1; |G| <= 0.3; still air.

```
$ transportation solve transport_vehicle_resistance "m=1500 kg" "v=27.77777778 m/s" "C_D=0.3 1"
"A=2.2 m^2"
F_roll = 220.725 N
F_aero = 305.5555556 N
F_grade = 0 N
F_total = 526.2805556 N
P = 14618.90432 W
P_kW = 14.61890432
  by: Vehicle resistance forces and power at a speed  (F = f_r m g cos(theta) + rho C_D A v^2/2 + m
g sin(theta), tan(theta) = G; P = F v)
  m = 1500 kg
  v = 27.77777778 m/s
  C_D = 0.3
  A = 2.2 m^2
  assumes: m, C_D, A, rho > 0; v >= 0; 0 <= f_r <= 0.1; |G| <= 0.3; still air
```

What the worked example means: This calculation determines the forces opposing a vehicle's motion at speed 27.7778 m/s. It sums rolling resistance of 220.725 N, aerodynamic drag of 305.556 N, and grade resistance of 0 N to find the total force of 526.281 N. The power needed to overcome these forces is then calculated as 14618.9 W. This result helps in understanding the energy demand for maintaining steady speed on a specific grade.

Questions and reports: support@ai999x.com. AI999X, The Future Is Self-Improving.